

Facto-DSA: A Cubic Factorisation Based Multivariate Digital Signature Algorithm

NGCC Public Key Cryptographic Algorithm Call

Borja Gómez
borgomez026@gmail.com

June 2026

Abstract

Facto-DSA is a factorisation-based multivariate digital signature algorithm. It hides an homogeneous quadratic invertible map by polynomial multiplication in $F_q[t]$. The public key is a dense system of homogeneous cubic equations. Inverting the map requires polynomial factorisation and solving the hidden quadratic invertible map. The field is chosen with $q \equiv 3 \pmod{4}$ so that square roots used in triangular inversion are computed by exponentiation. We write the central map that multiplies two polynomials:

$$(X, Y) \mapsto \mathcal{M}_n(Q(X) + R(Y), Y),$$

where $\mathcal{M}_n$ is ordinary multiplication of two degree bounded polynomials, $Q(X)$ an homogeneous quadratic triangular map and $R(Y)$ a random quadratic polynomial map. The public map is obtained by applying Isomorphism of Polynomials (IP2): the secret input map $(X, Y) = (L_1z, L_2z)$ and a secret full rank output map U .

Contents

1	Algorithm Description	3
1.1	Submission Notes	3
1.1.1	Algorithm identity	3
1.1.2	Relation to multivariate cryptography	3
1.2	Notation and conventions	4
1.3	Mathematical background	4
1.3.1	Quadratic forms and tensor representation	4
1.3.2	Tensor representation of polynomial multiplication	4
1.3.3	Bilinear forms induced by the secret input map	5
1.3.4	Polynomial factorisation and bounded degree splitting	5
1.4	Algorithm specification	6
1.4.1	Parameter tuple	6
1.4.2	Hashing and public key binding	6
1.4.3	Key generation	11
1.4.4	Signing	12
1.4.5	Verification	14
1.4.6	Correctness	15
2	Design Rationale	15
2.1	The Facto-DSA hard problem	15
2.2	Parameter selection	16
2.3	Failure probability and impact on security	17
2.4	Explanation of differences from related schemes	20
3	Security Statements and Analyses	21
3.1	Theoretical security	21
3.1.1	Signature experiment and advantage	21
3.1.2	Facto-DSA public inversion problem	21
3.1.3	Signing and bad events	22
3.1.4	Classical reduction	23
3.1.5	Quantum random oracle statement	24
3.1.6	Relation to the cubic map	24
3.2	Practical security	25
3.2.1	Direct algebraic solving	25
3.2.2	Attacks on factorisation based schemes	26
3.2.3	Cryptanalysis of the structure of the central map	34
3.2.4	MinRank and low rank attacks	35
3.2.5	One vector structural recovery	35
3.2.6	Complexity table for known classical and quantum attacks	36
3.3	Other security properties	36
4	Performance Evaluation	38
4.1	Performance analysis	38
4.1.1	Implementation Differences	39
4.1.2	Platform Differences	40
4.1.3	Transmission and storage cost	40
4.2	Performance test results	42
5	Feature Statements	57

1 Algorithm Description

1.1 Submission Notes

We present Facto-DSA for the NGCC public key algorithm call. The algorithm functionality is digital signature. The electronic package contains this specification, the reference and optimised implementations, scripts to test and measure running time metrics, known answer test vectors, basic information forms and intellectual property statements according to the public key algorithm submission requirements [1].

The algorithm is specified in [Section 1.4.3](#), [Section 1.4.4](#) and [Section 1.4.5](#). Failure probability and its security effect are stated in [Section 2.3](#). The difference from earlier factorisation schemes is stated in [Section 2.4](#). For the NGCC guide, [Section 1.4](#) gives the algorithm description, [Section 2](#) gives the design rationale, [Section 3](#) gives the security model and practical attack costs. The performance analysis and test results are stated in [Section 4](#) and [Section 5](#) gives the feature statements.

1.1.1 Algorithm identity

Algorithm name	Facto-DSA
Functionality	Digital signature
Public material	m homogeneous cubic equations in $r = 2n$ variables
Private material	Linear maps, output completion data, triangular Q and quadratic R
Central trapdoor	Factorisation of a product polynomial and inversion of triangular Q
Security model	EUFCMA in the random oracle model
Supported messages	Bit strings of length at most 2^{63} bits
Per key signing target	At least 2^{64} different messages

1.1.2 Relation to multivariate cryptography

In Multivariate public key cryptography we represent a public key, normally as a quadratic or cubic polynomial map

$$P = (P_1, \dots, P_m) : \mathbb{F}_q^r \longrightarrow \mathbb{F}_q^m.$$

A signature on a message digest $h \in \mathbb{F}_q^m$ is a preimage $z \in \mathbb{F}_q^r$ satisfying

$$P(z) = h.$$

The general public problem is polynomial system solving. A structured construction often has the form

$$P = T \circ F \circ S,$$

where F is an invertible with an easily invertible central map and S, T hiding its structure [7, 5, 6].

Facto-DSA follows this paradigm but uses a factorisation based central map. Earlier factorisation based constructions multiply two hidden polynomials and publish a quadratic map [15, 17]. Those quadratic variants were recently successfully cryptanalysed by techniques presented in [16] steps, having a structural flaw exploited by exterior algebra and MinRank style attacks. The submitted algorithm instead uses the cubic $Q(X) + R(Y)$ internal construction.

$$(X, Y) \mapsto (Q(X) + R(Y))Y,$$

where multiplication is polynomial multiplication in $\mathbb{F}_q[t]$. The extra quadratic part $R(Y)$, together with the triangular form of Q , was introduced after analysing new attacks on attacks on quadratic and cubic factorisation based scheme, as described in [Section 3.2.2](#).

1.2 Notation and conventions

Symbol	Meaning
q	prime field size, $q = 65519 \equiv 3 \pmod{4}$
n	factor vector length
D	product length, $D = 2n - 1$
r	public variable count, $r = 2n$
m	public equation count, with $n < m \leq D$
s	output minus dimension, $s = D - m$
S	secret input matrix in $\text{GL}_r(\mathbb{F}_q)$
L_1, L_2	first and second $n \times r$ block rows of S
U	secret output matrix in $\text{GL}_D(\mathbb{F}_q)$
T	public output projection matrix, the first m rows of U
Q	homogeneous triangular quadratic map $\mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$
R	homogeneous quadratic map $\mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$
$\mathcal{M}_n$	convolution tensor for multiplication in $\mathbb{F}_q[t]$
P	public cubic map $\mathbb{F}_q^r \rightarrow \mathbb{F}_q^m$
$R_{\max}$	maximum number of signing trials

Vectors are column vectors. For $a = (a_0, \dots, a_{n-1})^T \in \mathbb{F}_q^n$, the associated polynomial is

$$a(t) = \sum_{i=0}^{n-1} a_i t^i.$$

A polynomial of degree less than n is encoded by its coefficient vector padded to length n .

1.3 Mathematical background

1.3.1 Quadratic forms and tensor representation

For $V = \mathbb{F}_q^r$, a homogeneous quadratic form can be represented as

$$q(z) = z^T A z.$$

Since q is odd, the symmetric matrix representation is equivalent to the upper triangular monomial representation. A homogeneous cubic coordinate polynomial is stored in the ordered monomial basis

$$z_i z_j z_k, \quad 0 \leq i \leq j \leq k < r.$$

The number of cubic monomials is

$$N_3(r) = \binom{r+2}{3}.$$

The public key is stored as an $m \times N_3(r)$ coefficient table over $\mathbb{F}_q$.

1.3.2 Tensor representation of polynomial multiplication

Let

$$a(t) = \sum_{i=0}^{n-1} a_i t^i, \quad b(t) = \sum_{j=0}^{n-1} b_j t^j.$$

Their product has coefficient vector $c \in \mathbb{F}_q^D$, $D = 2n - 1$, where

$$c_\ell = \sum_{i+j=\ell} a_i b_j, \quad 0 \leq \ell \leq 2n - 2.$$

Define

$$\mathcal{M}_n \in \mathbb{F}_q^{D \times n^2}$$

by

$$(\mathcal{M}_n)_{\ell, in+j} = \begin{cases} 1, & \ell = i + j, \\ 0, & \text{otherwise.} \end{cases}$$

Then $\mathcal{M}_n(a \otimes b)$ is the coefficient vector of $a(t)b(t)$.

1.3.3 Bilinear forms induced by the secret input map

Let

$$S = \begin{pmatrix} L_1 \\ L_2 \end{pmatrix} \in \text{GL}_{2n}(\mathbb{F}_q), \quad L_1, L_2 \in \mathbb{F}_q^{n \times 2n}.$$

For a public variable vector $z \in \mathbb{F}_q^{2n}$, define

$$X = L_1 z, \quad Y = L_2 z.$$

Thus, we define the submitted central map as

$$\mathcal{F}_{Q,R}(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y) \in \mathbb{F}_q^D.$$

Then, we set the public map to

$$P(z) = T \mathcal{F}_{Q,R}(L_1 z, L_2 z),$$

where $T \in \mathbb{F}_q^{m \times D}$ has full row rank. In key generation, we sample a matrix $U \in \text{GL}_D(\mathbb{F}_q)$. The matrix T is set to the first m rows of U and U^{-1} is kept for signing. Thus, an output minus map is obtained. If $s = D - m > 0$, we complete a target $h \in \mathbb{F}_q^m$ to $(h, c) \in \mathbb{F}_q^D$ by choosing $c \in \mathbb{F}_q^s$ and applying U^{-1} .

1.3.4 Polynomial factorisation and bounded degree splitting

Let

$$f(t) = \sum_{i=0}^{D-1} y_i t^i \in \mathbb{F}_q[t], \quad D = 2n - 1.$$

As a result, we call a factorisation split valid when

$$f(t) = A(t)Y(t), \quad \deg A < n, \quad \deg Y < n.$$

If $f(t) \neq 0$, we factor f into irreducible factors and search for a subset of factor copies whose degree lies in the admissible interval

$$\max(0, \deg f - n + 1) \leq d \leq n - 1.$$

The selected factors form $A(t)$ and the remaining factors form $Y(t)$. Note that the factorisation split is not required to be balanced.

The $Q(X) + R(Y)$ central map gives an additional rescaling freedom. If

$$f(t) = A_0(t)Y_0(t),$$

then for every $\lambda \in \mathbb{F}_q^*$,

$$f(t) = (\lambda A_0(t))(\lambda^{-1} Y_0(t)).$$

Therefore, we use this freedom to search for λ such that

$$Q(X) = \lambda A_0 - R(\lambda^{-1} Y_0)$$

is solvable by the triangular inversion algorithm for Q .

1.4 Algorithm specification

1.4.1 Parameter tuple

For a particular Algorithm Instance, we specify a parameter tuple for Facto-DSA by

$$\text{param} = (q, n, r, D, m, s, R_{\max}),$$

where

$$q = 65519, \quad q \equiv 3 \pmod{4}, \quad r = 2n, \quad D = 2n - 1, \quad s = D - m, \quad n < m \leq D.$$

The congruence $q \equiv 3 \pmod{4}$ is required by the internal signing procedure. If $a \in \mathbb{F}_q$ is a quadratic residue, a square root is obtained as $a^{(q+1)/4}$. This rule is used inside triangular inversion, as further seen.

The value D identifies the degree of the product of two n degree secret polynomials, so $D = 2n - 1$. The value r is the number of variables, so $r = 2n$. The integer s is the output minus dimension. It gives the signer q^s possible completions of the public hash target before factorisation and thus increases the probability that a signing attempt factors into an internal target that can be inverted in the triangular form found in the signing procedure. The integer $R_{\max}$ bounds the number of full signing attempts, which is estimated in [Section 2.3](#).

The concrete algorithm instances parameter table is given in [Section 2.2](#).

1.4.2 Hashing and public key binding

Furthermore, we use the hash and XOF auxiliary functions required by the submission API package. The specification models them as random oracles in [Section 3.1](#). The domain labels are:

Label	Purpose
FDSA-H	message to field hashing

Let

$$\text{pkh} = H_{\text{pk}}(\text{Encode}(\text{pk}))$$

be the digest of the public key encoding. For a message M and public key digest pkh , define

$$h = \text{HashToField}_q^m(\text{FDSA-H} \parallel \text{pkh} \parallel M).$$

The value h is deterministic for a fixed public key and message. We obtain non deterministic signing by choosing an output completion $c \in \mathbb{F}_q^s$, choosing one of the possible factorisation candidates of the completed polynomial, choosing a nonzero rescaling factor and choosing one of the triangular inversion branches.

Auxiliary routines used by the algorithms The following table defines every routine name used by the pseudocode. The names don't correspond to those found in the implementation, but the same input and output relation is preserved. The auxiliary methods comprise the keygen, sign and verify needed internal routines for matrix and vector generation, encoding, decoding, factoring, completion, etc.

Routine	Meaning
$\text{XOF}(x)$	Expand the input byte string with the XOF function provided by the submission API.
$\text{ReadUInt16}(B)$	Read the next 16 bit unsigned word from stream B , in the byte order fixed by the implementation.
$\text{HashToField}_q^m(\ell, d)$	Convert a labelled byte string to m field elements by rejection from 16 bit words.
$\text{SampleFieldElement}(q)$	Sample one uniform element of $\mathbb{F}_q$.
$\text{SampleNonZeroFieldElement}(q)$	Sample one uniform element of $\mathbb{F}_q^*$.
$\text{SampleVector}(q, k)$	Sample one uniform vector in $\mathbb{F}_q^k$. If $k = 0$, return the empty vector.
$\text{SampleInvertibleMatrix}(q, k)$	Sample matrices in $\mathbb{F}_q^{k \times k}$ until an invertible one is obtained.
$\text{MatrixInverse}(A)$	Return A^{-1} , computed over $\mathbb{F}_q$.
$\text{SplitRows}(S, n, n)$	Return the first n rows and the last n rows of S .
$\text{FirstRows}(U, m)$	Return the first m rows of U .
$\text{UniformMatrix}(q, a, b)$	Sample a uniform matrix in $\mathbb{F}_q^{a \times b}$.
$\text{SampleHomogeneousQuadraticForm}(q, k)$	Sample a homogeneous quadratic form in k variables over $\mathbb{F}_q$.
$\text{SampleHomogeneousLinearForm}(q, k)$	Sample a homogeneous linear form in k variables over $\mathbb{F}_q$. If $k = 0$, return zero.
$\text{SampleHomogeneousQuadraticMap}(q, a, b)$	Sample b homogeneous quadratic forms in a variables.
$\text{SampleTriangularQuadraticMap}(q, n)$	Sample the triangular homogeneous map $Q : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ with nonzero leading square terms and triangular mixed terms.
$\text{FormalVariables}(r)$	Return formal variables $(z_0, \dots, z_{r-1})$.
$\text{CanonicalCubicOrder}(r)$	Return all triples (a, b, c) with $0 \leq a \leq b \leq c < r$, in lexicographic order.
$\text{CoefficientsCubic}(f, \mathcal{O})$	Return the coefficient vector of cubic form f in monomial order $\mathcal{O}$.
$\text{CubicMonomialVector}(z, \mathcal{O})$	Return $(z_a z_b z_c)_{(a,b,c) \in \mathcal{O}}$.
$\mathcal{M}_n(A, Y)$	Return the length $2n - 1$ coefficient vector of $A(t)Y(t)$.
$\text{ExpandCubicPublicKey}(\dots)$	Expand $T\mathcal{M}_n(Q(L_1z) + R(L_2z), L_2z)$ into cubic coefficient form.
$\text{EncodePublicKey}(\text{param}, C)$	Encode the parameter identifier and public cubic coefficient table.
$\text{DecodePublicKey}(\text{pk})$	Decode a public key into its parameter identifier and coefficient table, or reject.
$\text{EncodeSecretKey}(\dots)$	Encode the parameter identifier, S^{-1} , U^{-1} , Q , R and pkh .
$\text{DecodeSecretKey}(\text{sk})$	Decode the secret key fields used by signing, or reject.

Routine	Meaning
Encode(v), Decode $_q^k(b)$	Canonical field vector encoding and decoding. Decoding rejects values outside $\{0, \dots, q-1\}$.
$H_{\text{pk}}(b)$	Hash the canonical public key encoding to a fixed length public key digest.
Concat(a, b)	Return the vertical concatenation of vectors a and b .
VectorToPolynomial(y, D)	Return $\sum_{i=0}^{D-1} y_i t^i$.
FactorPolynomial(f)	Return the factorisation of nonzero $f \in \mathbb{F}_q[t]$.
BoundedFactorSplits(f, n)	Return all pairs (A_0, Y_0) with $f = A_0 Y_0$ and both degrees less than n .
SquareRoots(d)	Return all roots of $u^2 = d$ over $\mathbb{F}_q$, using $q \equiv 3 \pmod{4}$.
TriangularInvert(Q, W)	Return one X with $Q(X) = W$, or $\perp$.
BranchSchedule(B)	Put a branch list in the fixed order used by the implementation and apply any branch cap.
SelectBranch(B)	Select the branch returned by triangular inversion from the scheduled branch list.
ByteLength(b)	Return the number of bytes in byte string b .

Algorithm 1 SampleInvertibleMatrix(q, k)

Ensure: $A \in \text{GL}_k(\mathbb{F}_q)$

- 1: **repeat**
- 2: $A \leftarrow \text{UniformMatrix}(q, k, k)$
- 3: **until** $\det(A) \neq 0$
- 4: **return** A

Algorithm 2 SampleHomogeneousQuadraticMap(q, a, b)

Ensure: $G : \mathbb{F}_q^a \rightarrow \mathbb{F}_q^b$, homogeneous of degree two

- 1: **for** $j = 0$ to $b-1$ **do**
- 2: $G_j \leftarrow 0$
- 3: **for** $u = 0$ to $a-1$ **do**
- 4: **for** $v = u$ to $a-1$ **do**
- 5: $c \leftarrow \text{SampleFieldElement}(q)$
- 6: $G_j \leftarrow G_j + cX_uX_v$
- 7: **end for**
- 8: **end for**
- 9: **end for**
- 10: **return** $(G_0, \dots, G_{b-1})$

Algorithm 3 HashToField $_q^m$ (label, data)

Require: field size $q = 65519$, output length m , domain label and input data

Ensure: vector $h \in \mathbb{F}_q^m$

```
1:  $h \leftarrow ()$ 
2:  $B \leftarrow \text{XOF}(\text{label} \parallel \text{data})$ 
3: while  $|h| < m$  do
4:    $w \leftarrow \text{ReadUInt16}(B)$ 
5:   if  $w < q$  then
6:      $h \leftarrow h \parallel (w)$ 
7:   end if
8: end while
9: return  $h$ 
```

Algorithm 4 SampleTriangularQuadraticMap(q, n)

Ensure: triangular homogeneous quadratic map $Q : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$

```
1: for  $i = 0$  to  $n - 1$  do
2:    $a_i \leftarrow \text{SampleNonZeroFieldElement}(q)$ 
3:    $b_i \leftarrow \text{SampleHomogeneousLinearForm}(q, i)$ 
4:    $c_i \leftarrow \text{SampleHomogeneousQuadraticForm}(q, i)$ 
5:    $Q_i \leftarrow a_i X_i^2 + b_i(X_0, \dots, X_{i-1})X_i + c_i(X_0, \dots, X_{i-1})$ 
6: end for
7: return  $(Q_0, \dots, Q_{n-1})$ 
```

Algorithm 5 BoundedFactorSplits(f, n)

Require: nonzero polynomial $f \in \mathbb{F}_q[t]$, bound n

Ensure: list of pairs (A_0, Y_0) with $f = A_0 Y_0$, $\deg A_0 < n$, $\deg Y_0 < n$

```
1:  $\mathcal{F} \leftarrow \text{FactorPolynomial}(f)$ 
2:  $\mathcal{S} \leftarrow \emptyset$ 
3: for all submultisets  $E \subseteq \mathcal{F}$  do
4:    $A_0 \leftarrow \prod_{g \in E} g$ 
5:    $Y_0 \leftarrow f/A_0$ 
6:   if  $\deg A_0 < n$  and  $\deg Y_0 < n$  then
7:      $\mathcal{S} \leftarrow \mathcal{S} \cup \{(A_0, Y_0)\}$ 
8:   end if
9: end for
10: return  $\mathcal{S}$ 
```

Algorithm 6 SquareRoots(d) over $\mathbb{F}_q$

Require: $d \in \mathbb{F}_q$ **Ensure:** set of roots of $u^2 = d$

```
1: if  $d = 0$  then
2:   return  $\{0\}$ 
3: end if
4: if  $d^{(q-1)/2} \neq 1$  then
5:   return  $\emptyset$ 
6: end if
7:  $u \leftarrow d^{(q+1)/4}$ 
8: return  $\{u, -u\}$ 
```

Algorithm 7 TriangularInvert(Q, W)

Require: triangular map Q , target $W \in \mathbb{F}_q^n$ **Ensure:** one X with $Q(X) = W$, or $\perp$

```
1:  $\mathcal{B} \leftarrow \{()\}$ 
2: for  $i = 0$  to  $n - 1$  do
3:    $\mathcal{B}' \leftarrow \emptyset$ 
4:   for all  $x \in \mathcal{B}$  do
5:      $\beta \leftarrow b_i(x_0, \dots, x_{i-1})$ 
6:      $\gamma \leftarrow c_i(x_0, \dots, x_{i-1})$ 
7:      $\Delta \leftarrow \beta^2 - 4a_i(\gamma - W_i)$ 
8:      $\mathcal{R} \leftarrow \text{SquareRoots}(\Delta)$ 
9:     for all  $v \in \mathcal{R}$  do
10:       $u \leftarrow (-\beta + v)(2a_i)^{-1}$ 
11:       $\mathcal{B}' \leftarrow \mathcal{B}' \cup \{x \parallel (u)\}$ 
12:   end for
13: end for
14: if  $\mathcal{B}' = \emptyset$  then
15:   return  $\perp$ 
16: end if
17:  $\mathcal{B} \leftarrow \text{BranchSchedule}(\mathcal{B}')$ 
18: end for
19: return  $\text{SelectBranch}(\mathcal{B})$ 
```

Algorithm 8 ExpandCubicPublicKey($q, n, r, D, m, T, L_1, L_2, Q, R$)

Ensure: cubic coefficient table $C \in \mathbb{F}_q^{m \times \binom{r+2}{3}}$

```
1:  $(z_0, \dots, z_{r-1}) \leftarrow \text{FormalVariables}(r)$ 
2:  $X \leftarrow L_1 z$ 
3:  $Y \leftarrow L_2 z$ 
4:  $A \leftarrow Q(X) + R(Y)$ 
5:  $v \leftarrow T \mathcal{M}_n(A, Y)$ 
6: for  $j = 0$  to  $m - 1$  do
7:    $C[j, *] \leftarrow \text{CoefficientsCubic}(v_j, \text{CanonicalCubicOrder}(r))$ 
8: end for
9: return  $C$ 
```

Algorithm 9 KeyGen(param)

Require: param = $(q, n, r, D, m, s, R_{\max})$

Ensure: public key pk, secret key sk

- 1: $S \leftarrow \text{SampleInvertibleMatrix}(q, r)$
 - 2: $S^{-1} \leftarrow \text{MatrixInverse}(S)$
 - 3: $(L_1, L_2) \leftarrow \text{SplitRows}(S, n, n)$
 - 4: $U \leftarrow \text{SampleInvertibleMatrix}(q, D)$
 - 5: $U^{-1} \leftarrow \text{MatrixInverse}(U)$
 - 6: $T \leftarrow \text{FirstRows}(U, m)$
 - 7: $Q \leftarrow \text{SampleTriangularQuadraticMap}(q, n)$
 - 8: $R \leftarrow \text{SampleHomogeneousQuadraticMap}(q, n, n)$
 - 9: $C \leftarrow \text{ExpandCubicPublicKey}(q, n, r, D, m, T, L_1, L_2, Q, R)$
 - 10: $\text{pk} \leftarrow \text{EncodePublicKey}(\text{param}, C)$
 - 11: $\text{pkh} \leftarrow H_{\text{pk}}(\text{Encode}(\text{pk}))$
 - 12: $\text{sk} \leftarrow \text{EncodeSecretKey}(\text{param}, S^{-1}, U^{-1}, Q, R, \text{pkh})$
 - 13: **return** (pk, sk)
-

1.4.3 Key generation

Input: parameter tuple param and randomness source.

Output: public key pk and secret key sk.

1. Sample $S \leftarrow \text{GL}_{2n}(\mathbb{F}_q)$. If the sampled matrix is singular, resample.
2. Write

$$S = \begin{pmatrix} L_1 \\ L_2 \end{pmatrix}, \quad L_1, L_2 \in \mathbb{F}_q^{n \times 2n}.$$

The implementation stores S^{-1} , while we use L_1 and L_2 only to expand the public key during key generation.

3. Sample $U \leftarrow \text{GL}_D(\mathbb{F}_q)$. If the sampled matrix is singular, resample. Let $T \in \mathbb{F}_q^{m \times D}$ be the first m rows of U . Then $s = D - m$ rows are hidden from the public map and used only as completion coordinates during signing. Thus, we store U^{-1} .
4. Generate a homogeneous triangular quadratic map $Q = (Q_0, \dots, Q_{n-1})$, where

$$Q_i(X_0, \dots, X_i) = a_i X_i^2 + b_i(X_0, \dots, X_{i-1})X_i + c_i(X_0, \dots, X_{i-1}),$$

with $a_i \neq 0$. The coefficient b_i is a homogeneous linear form in the previous variables and c_i is a homogeneous quadratic form in the previous variables. As a result, every term of Q_i has total degree two. No stand alone linear term is used.

5. Generate a homogeneous quadratic map $R = (R_0, \dots, R_{n-1})$ from $\mathbb{F}_q^n$ to $\mathbb{F}_q^n$.
6. Define

$$\mathcal{F}_{Q,R}(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y).$$

7. Define the public map

$$P(z) = T\mathcal{F}_{Q,R}(L_1 z, L_2 z).$$

8. Expand each coordinate of P in the ordered cubic monomial basis $z_a z_b z_c$, $0 \leq a \leq b \leq c < 2n$ and store the resulting coefficient table as the public key.
9. Compute $\text{pkh} = H_{\text{pk}}(\text{Encode}(\text{pk}))$.
10. The secret key stores the data that we need for signing: S^{-1} , U^{-1} , the triangular coefficients of Q , the coefficients of R , pkh and the parameter identifier.

1.4.4 Signing

Algorithm 10 $\text{Sign}(\text{sk}, M)$

Require: secret key sk , message M

Ensure: signature σ or $\perp$

```

1:  $(\text{param}, S^{-1}, U^{-1}, Q, R, \text{pkh}) \leftarrow \text{DecodeSecretKey}(\text{sk})$ 
2:  $(q, n, r, D, m, s, R_{\max}) \leftarrow \text{param}$ 
3:  $S \leftarrow \text{MatrixInverse}(S^{-1})$ 
4:  $U \leftarrow \text{MatrixInverse}(U^{-1})$ 
5:  $(L_1, L_2) \leftarrow \text{SplitRows}(S, n, n)$ 
6:  $T \leftarrow \text{FirstRows}(U, m)$ 
7:  $h \leftarrow \text{HashToField}_q^m(\text{FDSA-H}, \text{pkh} \parallel M)$ 
8: for  $j = 0$  to  $R_{\max} - 1$  do
9:    $c \leftarrow \text{SampleVector}(q, s)$ 
10:   $y \leftarrow U^{-1} \text{Concat}(h, c)$ 
11:   $f \leftarrow \text{VectorToPolynomial}(y, D)$ 
12:  if  $f = 0$  then
13:    continue
14:  end if
15:   $\mathcal{S} \leftarrow \text{BoundedFactorSplits}(f, n)$ 
16:  for all  $(A_0, Y_0) \in \mathcal{S}$  do
17:    for all  $\lambda \in \mathbb{F}_q^*$  do
18:       $A \leftarrow \lambda A_0$ 
19:       $Y \leftarrow \lambda^{-1} Y_0$ 
20:       $W \leftarrow A - R(Y)$ 
21:       $X \leftarrow \text{TriangularInvert}(Q, W)$ 
22:      if  $X \neq \perp$  then
23:         $z \leftarrow S^{-1} \text{Concat}(X, Y)$ 
24:         $v \leftarrow T \mathcal{M}_n(Q(L_1 z) + R(L_2 z), L_2 z)$ 
25:        if  $v = h$  then
26:          return  $\text{Encode}(z)$ 
27:        end if
28:      end if
29:    end for
30:  end for
31: end for
32: return  $\perp$ 

```

Input: secret key $\mathbf{sk}$ and message M .

Output: signature σ or failure.

1. Parse $\mathbf{sk}$ and recover S^{-1} , U^{-1} , Q , R , $\mathbf{pkh}$ and the instance parameters.
2. Compute the deterministic target

$$h = \text{HashToField}_q^m(\text{FDSA-H} \parallel \mathbf{pkh} \parallel M).$$

3. For trial counter $j = 0, 1, \dots, R_{\max} - 1$:

- a) Choose a completion vector $c_j \in \mathbb{F}_q^s$. If $s = 0$, this vector is empty. Compute

$$y = U^{-1} \begin{pmatrix} h \\ c_j \end{pmatrix} \in \mathbb{F}_q^D.$$

Then $Ty = h$.

- b) Interpret $y = (y_0, \dots, y_{D-1})$ as

$$f_y(t) = \sum_{i=0}^{D-1} y_i t^i.$$

If $f_y = 0$, reject this trial.

- c) Factor $f_y(t)$ over $\mathbb{F}_q[t]$ and enumerate bounded degree factorisation splits

$$f_y(t) = A_0(t)Y_0(t), \quad \deg A_0 < n, \quad \deg Y_0 < n.$$

If no factorisation split is found, reject this trial.

- d) For each selected factorisation split and for the implementation rescaling schedule $\Lambda \subseteq \mathbb{F}_q^*$, set

$$Y = \lambda^{-1}Y_0, \quad A = \lambda A_0, \quad W = A - R(Y).$$

All polynomials are represented as coefficient vectors of length n .

- e) Try to solve the triangular system

$$Q(X) = W.$$

This is done by substituting the coordinates in order. Suppose that $X_0, \dots, X_{i-1}$ have already been chosen. Write

$$\beta_i = b_i(X_0, \dots, X_{i-1}), \quad \gamma_i = c_i(X_0, \dots, X_{i-1}).$$

The i -th equation becomes

$$a_i X_i^2 + \beta_i X_i + \gamma_i = W_i,$$

or

$$a_i X_i^2 + \beta_i X_i + (\gamma_i - W_i) = 0.$$

Its discriminant is

$$\Delta_i = \beta_i^2 - 4a_i(\gamma_i - W_i).$$

If Δ_i is not a square in $\mathbb{F}_q$, this branch fails. If Δ_i is a square, the roots are

$$X_i = \frac{-\beta_i \pm \sqrt{\Delta_i}}{2a_i}.$$

The two roots are appended as branch candidates. If all branches fail, try another rescaling, another factorisation split, or another completion.

f) If a solution X is found, compute

$$z = S^{-1} \begin{pmatrix} X \\ Y \end{pmatrix}.$$

Perform the internal check $P(z) = h$. If it passes, return

$$\sigma = \text{Encode}(z).$$

4. If all trials fail, return failure.

1.4.5 Verification

Algorithm 11 Verify(pk, M , σ)

Require: public key pk, message M , signature σ

Ensure: accept or reject

```

1: (param,  $C$ )  $\leftarrow$  DecodePublicKey(pk)
2: ( $q, n, r, D, m, s, R_{\max}$ )  $\leftarrow$  param
3: if ByteLength( $\sigma$ )  $\neq 2r$  then
4:   return reject
5: end if
6:  $z \leftarrow \text{Decode}_q^r(\sigma)$ 
7: if  $z = \perp$  then
8:   return reject
9: end if
10:  $\text{pkh} \leftarrow H_{\text{pk}}(\text{Encode}(\text{pk}))$ 
11:  $h \leftarrow \text{HashToField}_q^m(\text{FDSA-H}, \text{pkh} \parallel M)$ 
12:  $\mu \leftarrow \text{CubicMonomialVector}(z, \text{CanonicalCubicOrder}(r))$ 
13:  $w \leftarrow C\mu$ 
14: if  $w = h$  then
15:   return accept
16: end if
17: return reject

```

Input: public key pk, message M and signature $\sigma = \text{Encode}(z)$.

Output: accept or reject.

1. Reject if the signature length is not equal to $2r$ bytes.
2. Decode $z \in \mathbb{F}_q^r$ using the canonical field element encoding. Reject if any encoded field element is not in $\{0, \dots, q-1\}$.
3. Recompute $\text{pkh} = H_{\text{pk}}(\text{Encode}(\text{pk}))$.
4. Compute
$$h = \text{HashToField}_q^m(\text{FDSA-H} \parallel \text{pkh} \parallel M).$$
5. Evaluate the public cubic map $P(z)$ using the stored cubic coefficient table.
6. Accept if and only if $P(z) = h$.

1.4.6 Correctness

If signing returns a signature, then it has found a completion c , a completed vector

$$y = U^{-1} \begin{pmatrix} h \\ c \end{pmatrix}$$

such that $Ty = h$, a factorisation split

$$f_y(t) = A(t)Y(t),$$

and a solution of

$$Q(X) = A - R(Y).$$

As a result,

$$\mathcal{F}_{Q,R}(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y) = \mathcal{M}_n(A \otimes Y) = y.$$

The signer sets

$$z = S^{-1} \begin{pmatrix} X \\ Y \end{pmatrix},$$

so $L_1 z = X$ and $L_2 z = Y$. In consequence,

$$P(z) = T\mathcal{F}_{Q,R}(L_1 z, L_2 z) = Ty = h.$$

Verification recomputes the same deterministic h from (pkh, M) and accepts.

2 Design Rationale

2.1 The Facto-DSA hard problem

Public inversion problem For parameters (q, n, r, D, m) , let

$$P : \mathbb{F}_q^r \rightarrow \mathbb{F}_q^m$$

be a public Facto-DSA map. The direct inversion problem is:

$$\textbf{given } P \textbf{ and } h \in \mathbb{F}_q^m, \textbf{ find } z \in \mathbb{F}_q^r \textbf{ such that } P(z) = h.$$

This is a cubic polynomial system solving problem with a hidden factorisation based trapdoor.

Structure recovery problem A stronger attacker may attempt to recover equivalent private data. The structure recovery problem is:

$$\textbf{given } P, \textbf{ recover equivalent } S, U, Q, R$$

such that

$$P(z) = T\mathcal{M}_n((Q(L_1 z) + R(L_2 z)) \otimes L_2 z)$$

for all z . A successful structure recovery attack gives a signing trapdoor. The submitted construction is designed so that the known attacks (see [Section 3.2.2](#)) on the quadratic product construction, its projected variants and the simpler cubic construction $Q(X)Y$ do not recover such data.

2.2 Parameter selection

The required classical security levels are 128, 256 and 512 bits. The corresponding quantum resistant lower bounds are 80, 128 and 256 bits.

The criteria for choosing the parameters are as follows. First, the field is fixed as $q = 65519$, which gives a two byte field representation and satisfies $q \equiv 3 \pmod{4}$, as required by square root extraction of quadratic discriminants in triangular inversion. Second, n , m and s were selected so that direct algebraic solving, structural one vector search and known input separation recovery attacks stay above the required classical and quantum security levels. Third, the output minus value s was chosen so signing completion has always a high probabilistic success as seen in [Section 2.3](#). The classical and quantum complexity detailed cost formulae are stated in [Section 3.2](#) and the discarded attack chains are stated in [Section 3.2.2](#).

Table 2: Submitted Facto-DSA parameter sets and byte sizes.

Parameter	Facto-DSA-128	Facto-DSA-256	Facto-DSA-512
classical target	128	256	512
quantum target	80	128	256
q	65519	65519	65519
n	10	17	32
r	20	34	64
$D = 2n - 1$	19	33	63
$m = D - s$	13	32	62
s	6	1	1
$R_{\max}$	512	1024	2048
public key bytes	40,040	456,960	5,674,240
secret key bytes	3,094	11,662	61,922
signature bytes	40	68	128

To calculate the parameter cost, we continue by computing the public key size as

$$2m \binom{r+2}{3} \text{ bytes,}$$

because every public equation is stored as coefficients of all cubic monomials in r variables. Furthermore, the signature size is

$$2r \text{ bytes.}$$

In addition, we store S^{-1} , U^{-1} , the triangular coefficients of Q , the coefficients of R and a 32 byte public key digest in the secret key. The coefficient count used for the secret key is

$$N_Q(n) = \sum_{i=0}^{n-1} \left(1 + i + \binom{i+1}{2} \right) = n + \frac{n(n-1)}{2} + \frac{n(n-1)(n+1)}{6},$$

for the extended triangular map

$$Q_i = a_i X_i^2 + b_i(X_0, \dots, X_{i-1})X_i + c_i(X_0, \dots, X_{i-1}),$$

and

$$N_R(n) = n \binom{n+1}{2}$$

for the full homogeneous quadratic map $R : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$. The secret key byte count is thus

$$2(r^2 + D^2 + N_Q(n) + N_R(n)) + 32 \text{ bytes.}$$

This count stores every field element in two bytes and includes the public key digest. Then the suggested parametrisation for all the algorithm instances is given below:

2.3 Failure probability and impact on security

As seen in the algorithm description, the signing procedure is probabilistic, because the signer searches for a completed target that admits factorisation and triangular inversion. In the other hand, Keygen uses rejection sampling for satisfying the algorithm algebraic conditions and Verification is deterministic.

So it is left to us to estimate the failure probability on each phase of the algorithm.

Key generation failure Key generation samples two invertible matrices, $S \in \mathbb{F}_q^{r \times r}$ and $U \in \mathbb{F}_q^{D \times D}$. For a matrix sampled at random with the uniform law $k \times k$ matrix over $\mathbb{F}_q$,

$$\pi_k = \Pr[A \in \text{GL}_k(\mathbb{F}_q)] = \prod_{i=1}^k (1 - q^{-i}).$$

With unbounded rejection sampling, key generation has no external mathematical failure event and the expected number of samples is $1/\pi_k$ for each matrix. If an implementation imposes sampler bounds B_S and B_U , then

$$\Pr[\text{KeyGenFail}] \leq (1 - \pi_r)^{B_S} + (1 - \pi_D)^{B_U}.$$

Since

$$1 - \pi_k < q^{-1} + \frac{q^{-2}}{1 - q^{-1}},$$

matrix rejection affects key generation time only. The same convention is used for nonzero leading coefficients in the triangular map Q : the coefficient a_i is sampled from $\mathbb{F}_q^*$, or resampled until nonzero.

Verification failure Verification has no probabilistic failure event. This is, for every signature returned by the mathematical signing algorithm

$$\Pr[\text{Verify}(\text{pk}, M, \text{Sign}(\text{sk}, M)) = \text{reject}] = 0.$$

Invalid encodings and invalid signatures are rejected by definition, not by probability.

One signing attempt Fix pk and M . The hash target

$$h \in \mathbb{F}_q^m$$

is deterministic. A signing attempt chooses a completion $c \in \mathbb{F}_q^s$, computes

$$y = U^{-1} \begin{pmatrix} h \\ c \end{pmatrix} \in \mathbb{F}_q^D,$$

and interprets y as a polynomial

$$f_y(t) = \sum_{i=0}^{D-1} y_i t^i.$$

The attempt succeeds if there is a bounded factorisation split

$$f_y(t) = A_0(t)Y_0(t), \quad \deg A_0 < n, \quad \deg Y_0 < n,$$

and a rescaling $\lambda \in \Lambda \subseteq \mathbb{F}_q^*$ such that

$$Q(X) = \lambda A_0 - R(\lambda^{-1} Y_0)$$

has a triangular preimage. The returned candidate is then checked by public evaluation.

Bounded factorisation probability For a nonzero monic degree d polynomial, let its irreducible factor degree type be

$$\kappa = (\kappa_1, \dots, \kappa_d), \quad \sum_{e=1}^d e\kappa_e = d.$$

Let

$$I_q(e) = \frac{1}{e} \sum_{a|e} \mu(a) q^{e/a}$$

be the number of monic irreducible polynomials of degree e over $\mathbb{F}_q$. The number of monic degree d polynomials with factor degree type κ is

$$N_q(\kappa) = \prod_{e=1}^d \binom{I_q(e) + \kappa_e - 1}{\kappa_e}.$$

A bounded factorisation split exists if and only if the multiset containing κ_e copies of each degree e has a subset sum a satisfying

$$\max\{0, d - n + 1\} \leq a \leq n - 1.$$

Let $\mathbf{1}_{\text{split}}(\kappa)$ be the indicator of this condition. The exact monic degree d split probability is

$$p_{\text{split}}(n, q, d) = q^{-d} \sum_{\kappa: \sum e\kappa_e = d} N_q(\kappa) \mathbf{1}_{\text{split}}(\kappa).$$

For the completed vector $y \in \mathbb{F}_q^D$, the degree is not fixed. The completion factorisation probability is thus

$$p_{\text{split}}^{<D}(n, q) = \sum_{d=0}^{D-1} \frac{(q-1)q^d}{q^D} p_{\text{split}}(n, q, d).$$

For the submitted parameter sizes, the top degree $d = D - 1 = 2n - 2$ dominates and its subset condition is $a = n - 1$. Evaluating the above combinatorial formula gives the conservative lower bounds

Instance	n	D	$p_{\text{split}}^{<D}(n, q)$
Facto-DSA-128	10	19	≥ 0.26
Facto-DSA-256	17	33	≥ 0.22
Facto-DSA-512	32	63	≥ 0.19

Triangular inversion probability with rescaling The condition $q \equiv 3 \pmod{4}$ is used, after a residue test, each square root is computed as $d^{(q+1)/4}$. At step i , after $X_0, \dots, X_{i-1}$ are fixed, triangular inversion solves

$$a_i X_i^2 + b_i(X_0, \dots, X_{i-1})X_i + c_i(X_0, \dots, X_{i-1}) = W_i.$$

Write

$$\beta_i = b_i(X_0, \dots, X_{i-1}), \quad \gamma_i = c_i(X_0, \dots, X_{i-1}).$$

The next coordinate is found from

$$a_i X_i^2 + \beta_i X_i + (\gamma_i - W_i) = 0,$$

with discriminant

$$\Delta_i = \beta_i^2 - 4a_i(\gamma_i - W_i).$$

For a random right hand side, Δ_i is modelled as uniform in $\mathbb{F}_q$, since $a_i \neq 0$. In consequence, the number of roots has probabilities

$$\Pr[0] = \frac{q-1}{2q}, \quad \Pr[1] = \frac{1}{q}, \quad \Pr[2] = \frac{q-1}{2q}.$$

If all branches are followed, the branch count is the Galton Watson process

$$B_0 = 1, \quad B_{i+1} = \sum_{j=1}^{B_i} \xi_{i,j},$$

where $\xi_{i,j} \in \{0, 1, 2\}$ has the distribution above. With

$$g(x) = \frac{q-1}{2q} + \frac{x}{q} + \frac{q-1}{2q}x^2,$$

the survival probability through n triangular coordinates is

$$p_Q(n, q) = \Pr[B_n > 0] = 1 - g^{\circ n}(0).$$

For a rescaling set Λ of size N_λ , the probability that at least one rescaled right hand side has a triangular preimage is

$$p_\lambda(n, q, N_\lambda) = 1 - (1 - p_Q(n, q))^{N_\lambda}.$$

The submitted signing algorithm uses the full nonzero rescaling set $\Lambda = \mathbb{F}_q^*$, so $N_\lambda = q - 1$. For $q = 65519$, this makes p_λ equal to 1 up to a negligible term for all three submitted values of n .

Combined signing failure and retry bounds A conservative lower bound for one completion attempt is

$$p_{\text{trial}} \geq p_{\text{split}}^{<D}(n, q) p_\lambda(n, q, q - 1).$$

Using the lower bounds above and $p_\lambda \approx 1$, the retry bound required for a target failure level $2^{-\tau}$ is

$$R_{\min}(\tau) = \left\lceil \frac{\tau \log 2}{-\log(1 - p_{\text{trial}})} \right\rceil.$$

The submitted values choose a margin above $R_{\min}$:

Instance	τ	p_{trial} lower bound	$R_{\min}(\tau)$	$R_{\max}$	$(1 - p_{\text{trial}})^{R_{\max}}$
Facto-DSA-128	128	0.26	295	512	$< 2^{-222}$
Facto-DSA-256	256	0.22	715	1024	$< 2^{-367}$
Facto-DSA-512	512	0.19	1685	2048	$< 2^{-622}$

As a result, signing failure is far below the corresponding target security level in all the instances of Facto-DSA.

2.4 Explanation of differences from related schemes

Facto-DSA maintains the use of polynomial factorisation as the trapdoor but the submitted algorithm described in this document uses a cubic construction. Factorisation based schemes are found in works done by the author and others [15, 17, 16]. The attack chains on those schemes and on discarded quadratic Facto-DSA variants are developed in Section 3.2.2. Thus, we separate the submitted construction from those schemes to resist following cryptanalytic points.

1. X24 exposes a quadratic Hankel based structure. Its public space can be completed by a λ^3 biorthogonal construction, then rank one matrices are extracted by MinRank and the resulting data is transformed into a GRS key recovery problem [16].
2. Quadratic Facto-DSA mixes the two factors by a full input map, but the hidden antidiagonal input separation can still be recovered. Once the input separation is recovered, the Hankel space is recovered and the X24 style route applies with rank two elements rather than rank one elements.
3. Minus and input projection modifiers for quadratic Facto-DSA were analysed and broken in the tested regimes. For example, $c = 1$ is attacked by a hyperplane closure with λ^3 for $n \geq 5$ and $c = 2$ is attacked with λ^4 for $n \geq 6$.
4. The cubic construction $Q(X)Y$ was also analysed. Note that if Q is not homogeneous, lower degree terms reveal a quadratic matrix space. If Q is homogeneous triangular, the first derivative space still gives a public quadratic family from which the public attack chain recovers the hidden input separation and forges.
5. The cubic submitted $Q(X) + R(Y)$ construction for Facto-DSA changes the central map to $(Q(X) + R(Y))Y$. The term $R(Y)Y$ adds a Y only cubic component before the secret input map. We introduced this term to prevent the derivative route used against $Q(X)Y$ from isolating the same quadratic space.

3 Security Statements and Analyses

3.1 Theoretical security

Theoretical security claim and proofs for Facto-DSA are presented in this section. We follow the hash and sign style used in recent multivariate signature specifications, where a public map is treated as the verification key and signing is treated as trapdoor inversion of a hash target [2, 3]. We state the claim for existential unforgeability under chosen message attack. Furthermore, a quantum random oracle statement is given, since the NGCC guide encourages a proof in the quantum model.

3.1.1 Signature experiment and advantage

Let

$$\text{DSS} = (\text{KeyGen}, \text{Sign}^H, \text{Verify}^H)$$

be the Facto-DSA signature scheme. Let H be a random oracle with range $\mathbb{F}_q^m$. The input to H is the domain tag, the public key digest and the message:

$$H_{\text{pk}}(M) = H(\text{FDSA-H} \parallel \text{pkh} \parallel M).$$

The digest pkh is computed from the public key encoding. Then the hash is bounded to the target to the verification key.

Let $\mathcal{A}$ be an adversary. We define the EUF CMA experiment as follows.

$$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda), \quad (M^*, \sigma^*) \leftarrow \mathcal{A}^{H, \text{Sign}^H(\text{sk}, \cdot)}(\text{pk}).$$

The signing oracle records the set $\mathcal{Q}_S$ of signed messages. It returns

$$\sigma \leftarrow \text{Sign}^H(\text{sk}, M)$$

for each query M . We say that the adversary wins if

$$M^* \notin \mathcal{Q}_S$$

and

$$\text{Verify}^H(\text{pk}, M^*, \sigma^*) = 1.$$

The advantage is

$$\text{Adv}_{\text{DSS}}^{\text{EUF CMA}}(\mathcal{A}) = \Pr[\text{Verify}^H(\text{pk}, M^*, \sigma^*) = 1 \wedge M^* \notin \mathcal{Q}_S].$$

The probability is taken over key generation, the random oracle, signing choices and the coins of $\mathcal{A}$.

Following the EUF CMA convention, we define freshness on the message. Repeated signing of the same message may give different signatures, since signing is not deterministic. A new signature on a message that was already queried to the signing oracle is not counted as a forgery.

3.1.2 Facto-DSA public inversion problem

Given a parameter set $(q, n, r, D, m, s, R_{\max})$, let $\mathcal{P}_{\text{Facto-DSA}}$ be the public key distribution produced by KeyGen . A sampled public key is a cubic map

$$P : \mathbb{F}_q^r \longrightarrow \mathbb{F}_q^m$$

with hidden representation

$$P = \pi_m \circ U \circ \mathcal{F}_{Q,R} \circ S, \quad \mathcal{F}_{Q,R}(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y).$$

Here $S \in \text{GL}_r(\mathbb{F}_q)$, U is the full rank output map used before projection and $\pi_m : \mathbb{F}_q^D \rightarrow \mathbb{F}_q^m$ keeps the first m coordinates.

Thus, we define the public inversion problem by

$$P \leftarrow \mathcal{P}_{\text{Facto-DSA}}, \quad h^* \leftarrow \mathbb{F}_q^m.$$

An inverter receives (P, h^*) . It must output $z^* \in \mathbb{F}_q^r$ such that

$$P(z^*) = h^*.$$

Its advantage is

$$\text{Adv}_{\text{Facto-DSA}}^{\text{INV}}(\mathcal{B}) = \Pr[P(z^*) = h^* : P \leftarrow \mathcal{P}_{\text{Facto-DSA}}, h^* \leftarrow \mathbb{F}_q^m, z^* \leftarrow \mathcal{B}(P, h^*)].$$

For the proof with signing queries, we use a stronger target excluding inversion problem. The inverter is also given access to an oracle $\mathcal{O}_{\text{pre}}$. On input $h \neq h^*$, the oracle runs the private signing inversion routine and returns either a vector z with $P(z) = h$, or $\perp$. On input h^* , it returns $\perp$. The corresponding advantage is denoted

$$\text{Adv}_{\text{Facto-DSA}}^{\text{INV leak}}(\mathcal{B}).$$

This oracle is stronger than the real signing oracle, since targets in $\mathbb{F}_q^m$ may be chosen without first finding a message that hashes to them. As a result, the reduction to this problem is conservative.

Assumption 1 (Facto-DSA inversion with signing leakage) *For the submitted parameter sets, no feasible classical or quantum algorithm is known that, given (P, h^*) and target excluding access to $\mathcal{O}_{\text{pre}}$, outputs z^* with*

$$P(z^*) = h^*$$

below the claimed security level.

Note that this assumption is family specific. It does not state that the public map is a uniformly random cubic system. We state that inversion is hard for the submitted Facto-DSA public key distribution, even after many target excluding preimages have been seen.

3.1.3 Signing and bad events

Recall that for a message M , the target is

$$h = H_{\text{pk}}(M) = H(\text{FDSA-H} \parallel \text{pkh} \parallel M).$$

Verification first decodes σ . If decoding fails, verification rejects. If decoding succeeds, the decoded vector is unique:

$$z = \text{Decode}(\sigma) \in \mathbb{F}_q^r.$$

The verifier accepts only if

$$P(z) = H_{\text{pk}}(M).$$

As a result, any accepted fresh forgery is a public preimage of a hash target. we exclude the following events from the proof:

Bad_H : two different oracle inputs receive the same target,

Bad_{pk} : a public key digest collision is used,

Bad_{target} : a signing query asks for the programmed challenge target,

Bad_{unq} : verification accepts although the forged message was not queried to H ,

Bad_{sf} : a signing query fails because all trials fail.

Note that signing failure probability and its effect are analysed in [Section 2.3](#).

3.1.4 Classical reduction

Let $\mathcal{A}$ be a classical EUF CMA adversary. Assume that $\mathcal{A}$ makes at most $Q_H \geq 1$ random oracle queries and Q_S signing queries. An inverter $\mathcal{B}$ for [Assumption 1](#) is given (P, h^*) . It runs $\mathcal{A}$ on P , simulates the random oracle and uses $\mathcal{O}_{\text{pre}}$ to answer signing queries.

At the start, $\mathcal{B}$ chooses an index

$$i^* \leftarrow \{1, \dots, Q_H\}.$$

The i^* -th new random oracle query is answered by h^* . Every other new query is answered by an independent sample from $\mathbb{F}_q^m$. Repeated queries are answered from the same table.

For a signing query M , we let the reduction obtain the target $h = H_{\text{pk}}(M)$. If $h = h^*$, it aborts. If $h \neq h^*$, it queries $\mathcal{O}_{\text{pre}}(h)$ and returns the result to $\mathcal{A}$. This has the same distribution as the real signing oracle, except when the challenge target is requested.

If $\mathcal{A}$ outputs a valid fresh forgery (M^*, σ^*) and if M^* is the programmed oracle query, then

$$H_{\text{pk}}(M^*) = h^*.$$

Since verification accepts, $z^* = \text{Decode}(\sigma^*)$ is defined and satisfies

$$P(z^*) = h^*.$$

The inverter outputs z^* .

For at most $Q_H + Q_S + 1$ oracle inputs, the target collision bound is

$$\epsilon_H \leq \frac{(Q_H + Q_S + 1)^2}{2q^m}.$$

A signing query hits the challenge target with probability at most

$$\epsilon_{\text{target}} \leq \frac{Q_S}{q^m}.$$

If the forged message was never queried to the random oracle, acceptance requires guessing a fresh target for a fixed decoded vector and

$$\epsilon_{\text{unq}} \leq q^{-m}.$$

The probability that any signing query fails is bounded by

$$\epsilon_{\text{sf}} \leq Q_S p_{\text{sign fail}},$$

where $p_{\text{sign fail}}$ is given in [Section 2.3](#). If pkh has ℓ_{pkh} bits, then a fixed digest collision target has probability at most

$$\epsilon_{\text{pkh}} \leq 2^{-\ell_{\text{pkh}}}.$$

Theorem 1 (Classical EUF CMA security) *Let $\mathcal{A}$ be a classical adversary against Facto-DSA with at most $Q_H \geq 1$ random oracle queries and Q_S signing queries. Then there exists an inverter $\mathcal{B}$ for Facto-DSA inversion with signing leakage such that*

$$\text{Adv}_{\text{Facto-DSA}}^{\text{EUF CMA}}(\mathcal{A}) \leq Q_H \text{Adv}_{\text{Facto-DSA}}^{\text{INV leak}}(\mathcal{B}) + \epsilon_{\text{bad}},$$

where

$$\epsilon_{\text{bad}} \leq \frac{(Q_H + Q_S + 1)^2}{2q^m} + \frac{Q_S}{q^m} + q^{-m} + Q_S p_{\text{sign fail}} + 2^{-\ell_{\text{pkh}}}.$$

The running time of $\mathcal{B}$ is the running time of $\mathcal{A}$, plus random oracle table work, verification work and at most Q_S calls to $\mathcal{O}_{\text{pre}}$.

As a result, we obtain a public inversion algorithm from a fresh forgery in the random oracle model for the same Facto-DSA key distribution, with the signing leakage described above.

3.1.5 Quantum random oracle statement

In the quantum random oracle model, the adversary may query H in superposition. Signing queries remain classical, since the signature API receives ordinary byte strings. We follow the measure and reprogramme method used for full domain hash signatures and for probabilistic hash and sign with retry [4]. One quantum oracle query is measured and the measured input is reprogrammed to the inversion challenge.

Let $\mathcal{A}_Q$ be a quantum EUF CMA adversary with at most Q_H quantum random oracle queries and Q_S classical signing queries. Then a quantum inverter $\mathcal{B}_Q$ can be built for Facto-DSA inversion with signing leakage. The reduction has the usual quadratic loss in the number of quantum hash queries. In the notation used here, the statement is

$$\text{Adv}_{\text{Facto-DSA}}^{\text{QEUF CMA}}(\mathcal{A}_Q) \leq (2Q_H + 1)^2 \text{Adv}_{\text{Facto-DSA}}^{\text{QINV leak}}(\mathcal{B}_Q) + \epsilon_{\text{qbad}},$$

where ϵ_{qbad} contains the same signing failure, digest collision, target collision and unqueried target terms as in Theorem 1, with the standard quantum search loss for detecting programmed points.

3.1.6 Relation to the cubic map

Note that we do not replace the structured public key by a random cubic system in the proof. It keeps the public key distribution

$$P = \pi_m \circ U \circ \mathcal{F}_{Q,R} \circ S, \quad \mathcal{F}_{Q,R}(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y).$$

The reduction only says that a good forger would imply a good inverter. Since we assume no good inverter exists, we conclude no good forger exists and security depends on the hardness of public inversion or equivalent key recovery for this family.

A direct attack may solve

$$P(z) = h$$

as a system of m cubic equations in r public variables. A structural attack may try to recover data equivalent to (S, U, Q, R) . We analyse both cases in Section 3.2. The discarded constructions in Section 3.2.2 show why this distinction is needed.

3.2 Practical security

In this section we provide the complexity of known and classical and quantum attacks. These are: direct algebraic solving, structural attacks on discarded variants, MinRank and derivative methods, one vector recovery and general preimage search. These are further classified in a final table giving the classical and quantum cost for each submitted instance.

3.2.1 Direct algebraic solving

The direct attack solves

$$P(z) = h$$

as a public system of m cubic equations in the r public signature variables. The attack is conducted by computing the Grobner basis of the Ideal generated by the public equations $P(z) - h$, using the F4 algorithm [8, 9]. The observed solving degree estimate used for the submitted family is

$$d_{\text{solv}} \approx 2m + 1.$$

The cost proxy used in this specification is the Macaulay linear algebra cost at that degree,

$$C_{\text{GB}} = \binom{r + d_{\text{solv}}}{d_{\text{solv}}}^w, \quad d_{\text{solv}} = 2m + 1.$$

The computations uses $w = 3$, which is the conservative dense linear algebra value.

The quantities d_{solv} , d_{ff} and d_{lf} describe different moments in the Grobner basis computation. The value d_{ff} , or first fall degree, is the first degree at which a non-trivial reduction appears below the degree of regularity expected for a non-structured MQ system. It is a signal that the system starts to simplify algebraically in an useful way. The value d_{lf} , or last degree fall, is the last degree at which such a degree fall is observed before the computation reaches its final solving phase. By contrast, d_{solv} is the solving degree: it is the largest degree reached by the Gröbner basis computation before the system is fully reduced to a form from which the solution can be extracted. One may think of d_{ff} as the first important drop in degree during the computation, d_{lf} as the last observed drop and d_{solv} as the final degree that controls the overall algebraic cost. In general, a smaller d_{ff} suggests that useful relations appear earlier, whereas a larger d_{solv} usually indicates a harder system.

Table 3: Observed degree indicators for small Facto-DSA parameter examples.

n	m	r	d_{ff}	d_{lf}	d_{solv}
4	6	8	7	11	13
5	7	10	8	13	15
5	8	10	8	14	17
6	7	12	9	14	15

Starting with $n = 6, m = 7$ the computation was stopped at 100GB RAM consumption mark, after 6 hours of CPU processing, it didn't reach the last step of the computation in a modern computer. Knowing that typically $m = D - s$ it is naive to think that $m = n + 1$ as in $n = 6, m = 7$ case, but we selected $m = n + 1$ just for these computations to show that Facto-DSA exhibits $d_{\text{solv}} = 2m + 1$ and other degree fall conditions. Then with these observations we can estimate the classical bit in terms of time complexity of the F_4 algorithm.

Instance	variables r	equations m	d_{solv}	classical bits
Facto-DSA-128	20	13	27	129
Facto-DSA-256	34	32	65	265
Facto-DSA-512	64	62	125	512

For signature computation a completed target is selected which is a product of polynomials

$$f(t) = A(t)Y(t).$$

For a fixed nonzero value Y , the possible products with $\deg A < n$ form a set of size q^n inside the q^D coefficient space of degree less than $D = 2n - 1$. In consequence,

$$\Pr[f(t) \in Y(t) \cdot \mathbb{F}_q[t]_{<n}] = q^{n-D} = q^{-(n-1)}.$$

For a fixed completed target f , the set of right factors

$$\mathcal{Y}(f) = \{Y \in \mathbb{F}_q^n : \exists A \in \mathbb{F}_q^n, f(t) = A(t)Y(t)\}$$

has size at most $(q-1)N_{\text{div}}(f)$, where $N_{\text{div}}(f)$ is the number of bounded divisor choices of f . A random completed Y with minus output s hits a valid right factor with probability at most

$$\frac{q^s(q-1)N_{\text{div}}(f)}{q^n}.$$

Then for the hybrid attack one must guess k variables, having a total of q^k possible Grobner basis for polynomial systems in m equations in $n-k$ variables. This attack has a higher algebraic complexity, concluding that the best hybrid value is obtained at $k = 0$.

3.2.2 Attacks on factorisation based schemes

Recall that Facto-DSA is a cubic multivariate digital signature scheme, where the construction was thought to resist earlier candidate constructions cryptanalysis. Here we start enumerating the digital signature public key algorithms that have an equivalent signing method or a direct forgery. We use these results to explain why the submitted central map

$$F(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y)$$

was selected as the final candidate and why Facto-DSA is different from other studied candidates of the author and others.

For a vector space V and a symmetric matrix space $E \subseteq \text{Sym}(V)$, define

$$\Phi_{E,\ell} : V \otimes \Lambda^{\ell-1}V \longrightarrow E^* \otimes \Lambda^\ell V, \quad \Phi_{E,\ell}(x \otimes \omega)(M) = Mx \wedge \omega.$$

The public λ^ℓ closure is

$$E^{(\ell)} = \{N \in \text{Sym}(V) : Nx \wedge \omega = 0 \text{ for all } x \otimes \omega \in \ker \Phi_{E,\ell}\}.$$

This is a linear algebra operation on the public matrices. We use it to recover hidden matrix containers that are larger than the published output span.

Attack A: X24/X-1 work, separated product signatures Following the X24/X-1 setting, we write the central map as

$$F_A(a, b) = \mathcal{M}_n(a \otimes b), \quad (a, b) \in \mathbb{F}_q^n \times \mathbb{F}_q^n.$$

Equivalently, each target vector is the coefficient vector of the product $a(t)b(t)$. After the secret input map, the public key publishes a subspace of a disguised Hankel space

$$\mathcal{H}_S = \{S^T H(a) S : a \in \mathbb{F}_q^{2n-1}\}, \quad H(a)_{ij} = a_{i+j}.$$

Let $E \subseteq \mathcal{H}_S$ be the public quadratic matrix span.

1. *Exterior biorthogonal completion.* Then, we compute

$$K = E^{(3)}.$$

For the separated Hankel product, the closure equals the completed hidden space:

$$K = \mathcal{H}_S.$$

Thus, the output truncation is removed and the full Hankel container is recovered.

2. *Rank one extraction.* Solve

$$0 \neq M \in K, \quad \text{rank} M = 1.$$

Then, we use rank one elements of the form

$$M_v = S^T v v^T S,$$

where $[v]$ ranges over the rational normal curve

$$[1 : t : t^2 : \dots : t^{n-1}], \quad t \in \mathbb{F}_q \cup \{\infty\}.$$

3. *Coordinate recovery.* The recovered projective points form a disguised generalised Reed Solomon configuration. In addition, the rational normal curve is recovered by the standard GRS key recovery method, giving an equivalent change of coordinates that sends K to the ordinary Hankel space.
4. *Equivalent trapdoor.* After this change of coordinates we rewrite the public key as an equivalent multiplication map

$$(a, b) \mapsto a(t)b(t)$$

followed by a recovered output transformation.

5. *Forgery.* Given a target h , we solve the recovered output completion equation for a product vector w . We interpret w as a polynomial $f_w(t)$, factor it over $\mathbb{F}_q[t]$ and choose a bounded factorisation split

$$f_w(t) = a(t)b(t), \quad \deg a < n, \quad \deg b < n,$$

and map (a, b) back to the public coordinates. As a result, a signature z is obtained with $P(z) = h$.

As a result, the attack succeeds because the exterior closure gives the missing Hankel equations, rank one MinRank gives the rational normal curve and polynomial factorisation gives preimages for arbitrary completed targets.

Attack B: plain quadratic Facto-DSA The central map in the plain quadratic Facto-DSA setting is

$$F_B(X, Y) = \mathcal{M}_n(X \otimes Y), \quad (X, Y) \in \mathbb{F}_q^n \times \mathbb{F}_q^n.$$

The public map is

$$P(z) = T \mathcal{M}_n((L_1 z) \otimes (L_2 z)), \quad z \in V = \mathbb{F}_q^{2n}.$$

Let $S_F = (L_1; L_2)$. The public matrices span a subspace

$$E \subseteq S_F^T \mathcal{B}_H S_F,$$

where

$$\mathcal{B}_H = \left\{ \begin{pmatrix} 0 & H \\ H^T & 0 \end{pmatrix} : H \in H_n \right\}, \quad H_n = \{H(a) : H(a)_{ij} = a_{i+j}\}.$$

1. *Block Hankel completion.* Then, we compute

$$K = E^{(3)}.$$

For the plain quadratic construction,

$$K = S_F^T \mathcal{B}_H S_F.$$

As a result, the exterior closure recovers the whole block Hankel container even though the public equations are mixed by a full input map.

2. *Rank two MinRank.* We search inside K for

$$0 \neq M \in K, \quad \text{rank} M \leq 2.$$

The elements used in the attack are

$$M_v = S_F^T \begin{pmatrix} 0 & vv^T \\ vv^T & 0 \end{pmatrix} S_F, \quad 0 \neq v \in \mathbb{F}_q^n.$$

They have rank two and are the hyperbolic analogue of the rank one matrices of the separated X24 attack.

3. *Hyperbolic decomposition.* Each recovered rank two matrix has a decomposition

$$M_v = \alpha_v \beta_v^T + \beta_v \alpha_v^T,$$

where one covector belongs to the first hidden factor side and the other belongs to the second hidden factor side. The decomposition is public up to scalar and interchange.

4. *Input separation orientation.* We collect many pairs $\{\alpha_v, \beta_v\}$. The incidence rule separates the two families: two covectors from the same hidden side never form a hyperbolic pair, while every recovered rank two element contains the same one covector from each side. Thus, a direct sum decomposition is recovered

$$V^* = A^* \oplus B^*.$$

Dualizing gives the hidden factor separation in the input variables.

5. *Reduction to a separated product.* In a basis adapted to the recovered input separation, every public matrix has the form

$$M_i \sim \begin{pmatrix} 0 & G_i \\ G_i^T & 0 \end{pmatrix}.$$

As a result, the dense mixed public key has been reduced to a separated bilinear product key.

6. *Hankel recovery.* We find a symmetriser C_s such that

$$G_i C_s \in \text{Sym}_n(\mathbb{F}_q) \quad \text{for all } i.$$

Following this, the symmetrised block space is then completed by the ordinary λ^3 closure. Rank one extraction and rational normal curve recovery give an equivalent polynomial basis.

7. *Forgery.* With the recovered equivalent central map, we complete the output target, factor the product polynomial, choose a bounded factorisation split and map the factors back to public coordinates. The produced vector verifies under the original public key.

As a result, full input mixing does not prevent structural recovery in the quadratic construction. It changes the X24 rank one step into a rank two hyperbolic step, but the attack still reaches polynomial multiplication and then forges.

Attack C: quadratic Facto-DSA with minus and input projection The central map in this setting is still the quadratic product

$$F_C(X, Y) = \mathcal{M}_n(X \otimes Y),$$

but only a projected input space and a shortened output space are published. In matrix form, the projected quadratic variant used

$$Q_i = S^T \begin{pmatrix} 0 & H_i \\ H_i^T & 0 \end{pmatrix} S, \quad S \in \mathbb{F}_q^{2n \times (2n-c)}.$$

Let $V = \mathbb{F}_q^{2n-c}$. The two hidden projected factor maps are

$$L_1, L_2 : V \rightarrow \mathbb{F}_q^n.$$

Thus, we define the two kernel sections

$$K_A = \ker L_2, \quad K_B = \ker L_1.$$

For a general projection, $\dim K_A = \dim K_B = n - c$. The true projected block symmetric container is

$$C_{\text{true}} = S^T \left\{ \begin{pmatrix} 0 & G \\ G^T & 0 \end{pmatrix} : G \in \text{Sym}_n(\mathbb{F}_q) \right\} S.$$

1. *Higher exterior completion.* Then, we compute

$$C = E^{(\ell)}.$$

The successful closures are $\ell = 3$ for $c = 1$ and $\ell = 4$ for $c = 2$. In those cases,

$$C = C_{\text{true}}.$$

The exterior degree increases because projection hides more of the block container.

2. *Polar rank defect.* For $z \in V$, we define

$$\pi_z : C \rightarrow V^*, \quad M \mapsto Mz.$$

If $z \in K_A$, then $L_2 z = 0$. In hidden coordinates this kills one side of every block matrix, so $\text{rank} \pi_z$ drops. The same holds for $z \in K_B$. In consequence, the rank defect locus of π_z contains

$$K_A \cup K_B.$$

Sampling defect vectors and taking their span recovers

$$H = K_A + K_B.$$

3. *Separating the recovered container.* We restrict the recovered container to H . Then we solve the anti commutant equations

$$J^T M + M J = 0 \quad (M \in C|_H).$$

In hidden coordinates, J acts as $+1$ on one projected factor section and -1 on the other. Since q is odd, the eigenspaces of J recover

$$H = K_A \oplus K_B$$

up to interchange.

4. *Quotient augmentation.* Let G_0 be a public complement of H , so

$$V = H \oplus G_0, \quad \dim G_0 = c.$$

The recovered restrictions determine all block entries involving K_A or K_B . What remains is the quotient block on G_0 .

For $c = 1$, we choose a generator g of G_0 . If g has hidden components (a_0, b_0) , then public values determine

$$u^T Q_i g \quad (u \in H), \quad g^T Q_i g = 2a_0^T G_i b_0.$$

As a result, the missing quotient row, column and diagonal scalar of each mixed block are determined.

For $c = 2$, we choose a basis (g_1, g_2) of G_0 . Quadratic evaluations determine only the symmetric part of the 2×2 quotient block. The alternating part is invisible to evaluations of $z^T M z$ on a two dimensional quotient. As a result, write

$$K_i(\tau_i) = K_i^{(0)} + \tau_i J_2, \quad J_2 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}.$$

This $\tau_i J_2$ term is the skew ambiguity. It is recovered by imposing the existence of a common symmetriser. We guess a constant number of seed skew values until an invertible C_s satisfies the symmetriser equations for the seed equations. Then each remaining τ_i is determined by linear algebra by

$$\text{skew}((K_i^{(0)} + \tau_i J_2)C_s) = 0.$$

5. *Hankel completion and factor recovery.* After quotient augmentation, we obtain completed mixed blocks K_i . We find C_s such that

$$K_i C_s \in \text{Sym}_n(\mathbb{F}_q) \quad \text{for all } i.$$

Then we apply λ^3 to recover the Hankel space, extract rank one matrices and recover the rational normal curve coordinates.

6. *Forgery.* We complete the public target, factor the recovered product polynomial and use the scalar freedom

$$(a, b) \sim (\lambda a, \lambda^{-1} b)$$

to satisfy the projected quotient constraints. Mapping the completed factors back through the recovered input map gives z with $P(z) = h$.

As a result, minus and input projection were not retained as a repair of quadratic Facto-DSA.

Attack D: the simpler cubic map $Q(X)Y$ The central map in the simpler cubic setting is

$$F_D(X, Y) = \mathcal{M}_n(Q(X) \otimes Y),$$

where $Q = (Q_0, \dots, Q_{n-1})$ is an extended triangular homogeneous quadratic map. The public map is

$$P(z) = T\mathcal{M}_n(Q(L_1 z) \otimes L_2 z).$$

In hidden variables,

$$X = L_1 z, \quad Y = L_2 z, \quad F_D(X, Y) = \mathcal{M}_n(Q(X) \otimes Y).$$

As a result, we rejected this construction because a public derivative space exposes the first kernel, then a second public linear system exposes the other kernel. The attack then reaches an equivalent polynomial multiplication basis and a recovered triangular inverse for Q .

1. *Derivative quadratic space.* We compute the derivative as

$$DF_{(X,Y)}(\Delta X, \Delta Y) = \mathcal{M}_n(DQ(X)[\Delta X] \otimes Y) + \mathcal{M}_n(Q(X) \otimes \Delta Y).$$

As a result, every first partial derivative of a public cubic coordinate is a public quadratic form. Let

$$\mathcal{D} = \left\langle \frac{\partial P_i}{\partial z_j} : 0 \leq i < m, 0 \leq j < 2n \right\rangle \subseteq \text{Sym}_{2n}(\mathbb{F}_q).$$

In hidden coordinates, the forms in $\mathcal{D}$ have only X^2 and XY terms and no Y^2 terms.

2. *Why K_1 is visible.* We set

$$K_1 = \ker L_1 = \{z : X = 0\}.$$

Since every derivative quadratic form contains at least one X factor, every $M \in \mathcal{D}$ satisfies

$$u^T M v = 0 \quad (u, v \in K_1).$$

As a result, K_1 is a common totally isotropic subspace of the derivative space. The polar maps of $\mathcal{D}$ also preserve this subspace.

3. *Kipnis Shamir recovery of K_1 .* Choose an invertible matrix $M_0 \in \mathcal{D}$ and form public endomorphisms

$$A_M = M_0^{-1} M, \quad M \in \mathcal{D}.$$

The hidden subspace K_1 is a common invariant subspace:

$$A_M K_1 \subseteq K_1.$$

For a random linear combination A of the A_M , factor

$$\chi_A(x) = \prod_j f_j(x).$$

For each product $g(x) = \prod_{j \in J} f_j(x)$ with $\deg g = n$, form

$$U_g = \ker g(A).$$

The candidate is kept only if

$$A_M U_g \subseteq U_g \quad \text{for several } M, \quad u^T N v = 0 \quad (u, v \in U_g, N \in \mathcal{D}).$$

The surviving n dimensional subspace is K_1 .

4. *Recovery of K_2 .* Let

$$K_2 = \ker L_2 = \{z : Y = 0\}.$$

Since $F_D(X, 0) = 0$, the full subspace K_2 lies in the public zero set. After K_1 is known, choose a public complement C_0 with

$$V = K_1 \oplus C_0.$$

Every complement transverse to K_1 can be written as

$$K(A) = \{c + A(c) : c \in C_0\}, \quad A : C_0 \rightarrow K_1.$$

Adding a vector from K_1 changes only the hidden Y coordinate. Since $Q(X)Y$ is linear in Y , the equations

$$P(c + A(c)) = 0$$

are linear in the entries of A when they are evaluated on enough public sample points $c \in C_0$. Solving this public linear system gives $K_2 = K(A)$.

5. *Separation rewrite.* We choose bases B_X of K_2 and B_Y of K_1 and substitute

$$z = B_X x + B_Y y.$$

The public equations become only $x^2 y$ forms:

$$P_i(x, y) = \sum_{a=0}^{n-1} \sum_{b=0}^{n-1} G_{i,ab} q_a(x) y_b.$$

Here q_a are quadratic forms in x and $G_i = (G_{i,ab})$ is an $n \times n$ matrix over $\mathbb{F}_q$.

6. *Isolation of the quadratic Q space.* We obtain the quadratic space from the coefficients of the public equations with respect to the recovered y variables. For each public equation and each y_b , we compute

$$q_{i,b}(x) = \left. \frac{\partial P_i(x, y)}{\partial y_b} \right|_{y=0}.$$

All $q_{i,b}$ belong to the same hidden output span

$$\mathcal{Q} = \langle q_0, \dots, q_{n-1} \rangle.$$

Then, we select a basis $(q_0, \dots, q_{n-1})$ by row reduction on the coefficient vectors of the quadratic forms. Then each coefficient form is written in that basis:

$$q_{i,b}(x) = \sum_{a=0}^{n-1} G_{i,ab} q_a(x).$$

Thus, we define the public matrix G_i from the scalars $G_{i,ab}$. As a result, we recover both the quadratic space $\mathcal{Q}$ and the published multiplication matrix space

$$\mathcal{G} = \langle G_0, \dots, G_{m-1} \rangle.$$

7. *Recovery and completion of the G matrix space.* The matrices G_i are a disguised truncated multiplication space. We find an invertible symmetriser C_s by solving the public linear equations

$$G_i C_s = (G_i C_s)^T \quad (0 \leq i < m).$$

Following symmetrisation, we set

$$E_i = G_i C_s.$$

Then $E = \langle E_i \rangle$ is a public subspace of a disguised Hankel space. The λ^3 closure gives the completed space

$$E^{(3)} = \{U^T H(a) U : a \in \mathbb{F}_q^{2n-1}\}.$$

Rank one elements of this completed space give a disguised rational normal curve. Recovering that curve gives an equivalent polynomial coordinate system. The inverse conversion $G = EC_s^{-1}$ gives a full $D = 2n - 1$ dimensional set of multiplication matrices. The published target equation is then a projection of this completed system, so a target $h \in \mathbb{F}_q^m$ can be completed by solving a public linear system for $w \in \mathbb{F}_q^D$.

8. *Recovery of the triangular flag of $\mathcal{Q}$.* The space $\mathcal{Q}$ is equivalent to the output span of a triangular quadratic map. Let the unknown hidden input coordinates be ordered so that

$$Q_i(X_0, \dots, X_i) = a_i X_i^2 + b_i(X_0, \dots, X_{i-1})X_i + c_i(X_0, \dots, X_{i-1}).$$

Then, we recover the flag from rank one quotients. We start with $V_0 = \mathbb{F}_q^n$. At stage i , we restrict every form in $\mathcal{Q}$ to V_i . In the correct hidden coordinates, one linear combination restricts to a rank one square term on the quotient. In consequence, solve

$$0 \neq M_i \in \mathcal{Q}|_{V_i}, \quad \text{rank } M_i = 1.$$

Then we define

$$V_{i+1} = \ker M_i \subset V_i.$$

Thus, this gives a chain

$$\mathbb{F}_q^n = V_0 \supset V_1 \supset \dots \supset V_n = 0, \quad \dim V_i = n - i.$$

A complement of V_{i+1} in V_i gives the recovered coordinate X_i , up to scale. The same linear combinations give an equivalent triangular output basis. After n steps, we rewrite $\mathcal{Q}$ as an extended triangular map of the form above.

9. *Inversion of the recovered triangular map.* Given a target $W = (W_0, \dots, W_{n-1})$, we solve the recovered triangular map coordinate by coordinate. Once $X_0, \dots, X_{i-1}$ are known, we set

$$\beta_i = b_i(X_0, \dots, X_{i-1}), \quad \gamma_i = c_i(X_0, \dots, X_{i-1}).$$

Then X_i is obtained from

$$a_i X_i^2 + \beta_i X_i + (\gamma_i - W_i) = 0.$$

The discriminant is

$$\Delta_i = \beta_i^2 - 4a_i(\gamma_i - W_i).$$

If Δ_i is a square, the roots are

$$X_i = \frac{-\beta_i \pm \sqrt{\Delta_i}}{2a_i}.$$

The condition $q \equiv 3 \pmod{4}$ gives $\sqrt{\Delta_i} = \Delta_i^{(q+1)/4}$ for square Δ_i . If a branch fails, we try another completion, factorisation split, or scalar λ .

10. *Forgery.* Given h , we solve the completed output equation and factor the recovered product polynomial:

$$f_w(t) = u(t)v(t), \quad \deg u < n, \quad \deg v < n.$$

For $\lambda \in \mathbb{F}_q^*$, solve

$$Q(x) = \lambda u$$

with the recovered triangular inverse and set

$$y = \lambda^{-1}v.$$

The vector (x, y) is mapped back through the recovered separation basis. The public evaluation confirms $P(z) = h$.

Note that if Q is not homogeneous, lower degree terms expose an even simpler quadratic matrix subspace. If Q is homogeneous, the derivative space still exposes K_1 , which starts the full recovery chain. As a result, $Q(X)Y$ was rejected.

3.2.3 Cryptanalysis of the structure of the central map

For the submitted setting, we write the central map as

$$F_E(X, Y) = \mathcal{M}_n(Q(X) + R(Y), Y).$$

The public map is

$$P(z) = T\mathcal{M}_n((Q(L_1z) + R(L_2z)) \otimes L_2z).$$

In hidden variables,

$$F_E(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y).$$

Its derivative is

$$DF_{(X,Y)}(\Delta X, \Delta Y) = \mathcal{M}_n(DQ(X)[\Delta X] \otimes Y) + \mathcal{M}_n(DR(Y)[\Delta Y] \otimes Y) + \mathcal{M}_n((Q(X) + R(Y)) \otimes \Delta Y).$$

The two terms involving R add Y^2 terms to the derivative quadratic space. As a result, the derivative forms are no longer of pure $X^2 + XY$ type and $K_1 = \ker L_1$ is no longer a common totally isotropic subspace. The Kipnis Shamir start of Attack D is thus removed.

Furthermore, the term $R(Y)Y$ changes the recovery of K_2 . The zero subspace $K_2 = \ker L_2$ still satisfies $F(X, 0) = 0$, but after a candidate K_1 is not available, there is no public linear complement parametrisation of K_2 . Moreover, in a parametrisation $K(A) = \{c + A(c) : c \in C_0\}$, the equations

$$P(c + A(c)) = 0$$

contain cubic terms in the entries of A coming from $R(Y)Y$. Thus, this is the opposite of the linear recovery used against $Q(X)Y$.

The signing trapdoor is preserved. After output completion and factorisation the signer has

$$f_y(t) = A(t)Y(t).$$

Then, we compute

$$W = A - R(Y)$$

and solves

$$Q(X) = W$$

with the triangular inverse. As a result, R is a nonlinear modifier that changes the public derivative geometry while preserving owner inversion.

As a result, quadratic Facto-DSA is discarded because exterior algebra recovers a block Hankel space; projected quadratic Facto-DSA is discarded because higher exterior closures, polar defect splitting, quotient augmentation and skew completion recover the missing structure; the simpler cubic $Q(X)Y$ is discarded because its derivative space recovers K_1 , then K_2 , then a Hankel product basis. The submitted $Q(X) + R(Y)$ central map blocks the derivative separation used in the last chain while retaining the factorisation based signing algorithm.

3.2.4 MinRank and low rank attacks

Classical MinRank attacks attempt to recover hidden low rank linear combinations of public quadratic matrices [10, 11]. In a cubic scheme the public equations are not matrices, but every first derivative is a quadratic form. As a result, MinRank and support minors attacks apply to derivative spaces and auxiliary linearisations. There have been other cubic schemes that have been attacked by the presence of low rank invariants in the differential structure as in [12, 13].

Note that the discarded $Q(X)Y$ construction is vulnerable because its derivative space has the hidden type $X^2 + XY$ and no Y^2 part. This gives a common totally isotropic subspace K_1 and enables the Kipnis Shamir recovery described in Section 3.2.2.

In our submitted map

$$F(X, Y) = \mathcal{M}_n((Q(X) + R(Y)) \otimes Y),$$

the derivative contains

$$\mathcal{M}_n(DR(Y)[\Delta Y] \otimes Y) \quad \text{and} \quad \mathcal{M}_n(R(Y) \otimes \Delta Y),$$

which add the missing Y^2 component. The derivative space no longer has the same oil like invariant subspace. At the time of this specification, no complete MinRank, support minors, or derivative invariant recovery is known for the submitted instances.

3.2.5 One vector structural recovery

In the study by Pébereau for UOV they showed that, for UOV type systems, once a single vector in the hidden oil subspace is known, the secret subspace can often be recovered by linear algebra [14]. Facto-DSA is not UOV based but it contains an analogous hidden zero subspace

$$K_2 = \ker L_2.$$

Indeed, for every $z \in K_2$, one has $Y = L_2 z = 0$ and thus

$$P(z) = T\mathcal{M}_n((Q(L_1 z) + R(0)) \otimes 0) = 0.$$

As a result, a nonzero vector in K_2 is valuable structural information.

If an adversary guesses $0 \neq v \in K_2$, then the Jacobian of the public map at v is

$$DP_v(\delta z) = T\mathcal{M}_n(Q(L_1 v) \otimes L_2 \delta z),$$

because $L_2 v = 0$ and R is homogeneous. If multiplication by $Q(L_1 v)$ followed by T has rank n , then

$$\ker DP_v = K_2.$$

Then, one vector in K_2 can reveal the whole zero subspace by public linear algebra. For this reason the conservative structural brute force attack is the search for one nonzero vector in K_2 .

For a random vector in $\mathbb{F}_q^{2n}$,

$$\Pr[z \in K_2 \setminus \{0\}] = \frac{q^n - 1}{q^{2n} - 1} \approx q^{-n}.$$

The classical cost of this structural search is thus q^n and the Grover cost is $q^{n/2}$. This gives the intended quantum resistant levels:

Instance	n	classical one vector bits	Grover one vector bits
Facto-DSA-128	10	160	80
Facto-DSA-256	17	272	136
Facto-DSA-512	32	512	256

The table is stated in the field element cost model, where one element of $\mathbb{F}_q$ is counted as a 16 bit search unit. This is the most relevant brute force structural attack considered in the parameter selection.

3.2.6 Complexity table for known classical and quantum attacks

General preimage search for a random target $h \in \mathbb{F}_q^m$ has classical cost about q^m public evaluations and Grover cost about $q^{m/2}$. This is larger than the one vector structural search for all submitted instances. A signature reveals one public preimage of one hash target. The security assumption includes access to many such signatures. In addition, the hash input includes the public key digest and message and every returned signature is checked before output.

The following table summarises the estimates. The direct algebraic row uses the m -equation, r -variable model from this subsection with $d_{\text{solv}} = 2m + 1$ and $w = 3$. The one vector rows give the conservative structural brute force estimate from the previous subsection. The general preimage rows are included for comparison.

Complexity measure	Facto-DSA-128	Facto-DSA-256	Facto-DSA-512
Target classical security	128	256	512
Target quantum security	80	128	256
Direct algebraic solving, classical	129	265	512
One vector structural search, classical	160	272	512
One vector structural search, Grover	80	136	256
General preimage search, classical	208	512	992
General preimage search, Grover	104	256	496

All entries are given as bit complexities. The direct algebraic estimate is the lowest listed classical cost for the 128 and 256 bit instances and it is tied with the one vector cost for the 512 bit instance. As a result, the submitted instances meet the stated classical targets under the $d_{\text{solv}} = 2m + 1$ model. The quantum rows are given for search based attacks where Grover type speedups apply.

3.3 Other security properties

Note that timing variation during verification does not leak secret key material as only works with public material: the public key and the signature. Also evaluates the public map and comparing the result with the hash derived target.

Signing and key generation use secret data. The submitted reference, performance and resource implementations are intended to show correctness, portability and to measure the performance profile. They are not claimed to be constant time. In particular, signing works with variable time operations such as: target completion, polynomial factorisation, factor enumeration, rescaling search, square root tests, root selection and triangular quadratic inversion. For a future work, a hardened implementation should audit the following: finite field arithmetic, table accesses, branches, memory allocation, rejection loops, factorisation, root selection, cache performance and buffer clearing. Temporary buffers that contain secret key material, intermediate secret derived values and secret linear images should be cleared before return.

When signing, reusing the same long term key for many signatures is part of the intended use of the scheme. Recall that for any given message M there exists q^s completions where $s = 1$ for 256 and 512 bit instances. That gives enough randomness for choosing a valid signature path, which has been probed probabilistically that such signature point exists.

For multi key use, the hash input contains a digest of the public key together with the parameter set domain separation. Thus, the target value for one public key is different from the target value for another public key, except under the hash collision probability scenario already covered in the EUF-CMA analysis. A forgery under one public key does not translate into a forgery under another public key unless the adversary solves the public inversion problem for that second key or finds a relevant digest collision.

A fault during target completion, factorisation, split selection, rescaling, triangular inversion, or the final secret linear map could cause a malformed candidate signature. Such a value may leak information if it is returned directly. For this reason, the implementation performs public verification on every candidate signature before output.

4 Performance Evaluation

It is important to separate the performance analysis of all the submitted implementations in two parts: Analysis and test results.

4.1 Performance analysis

First, the analysis conducted here takes into account the documents released by NICCS. These are the "Guidelines for Self-Assessment of Cryptographic Algorithms Performance on x86 Architecture", same for ARM. These are found in:

<https://niccs.org.cn/niccs/Notice/2066798866453762048/WPSGH11R.pdf>

<https://niccs.org.cn/niccs/Notice/2066798866453762048/m31gM4bv.pdf>

These guidelines recommend measuring technical indicators by testing the three submitted implementations: Reference Implementation, Performance Optimization Version and Resource Optimization Version. A test bench project (ngcc_bench) is recommended, but it is not mandatory. It helps measuring requested indicators like: Dynamic/Static Memory consumed, cycles per operation, cycle timings, environment details (CPU, compiler, OS, RAM).

The test bench implementation repository can be found at:

https://github.com/openHiTLS/ngcc_bench

The source code of ngcc_bench has been modified to retrieve CPU cycles in MacOS Apple and RISC-V platforms. When measuring an algorithm instance of Facto-DSA, the tool generates a JSON report with the output tests done for each phase mode (Correctness, Memory and Performance). All implementations (Reference, Performance and Resource Optimised) compile into a library .so file, then ngcc_bench loads the algorithm .so library file to compute and retrieve all the measurements requested by NICCS.

A simple bash script (run-bench.sh) compiles all implementations into .so library files with the naming: libfactodsaX-Y.so where X is the target level bit (i.e 128,256,512) and Y is the name of the implementation, namely (ref,opti,res) for the Reference Implementation, Performance Optimization Version and Resource Optimization Version.

The same script handles the compilation in the running platform, if its x86-64, MacOS, ARM/AARCH64 or RISC-V then compiles the test bench program (ngcc_bench) and all the libraries .so of the algorithm instances. Then for each implementation version, it invokes the bench platform ngcc_bench and saves the results into these folders:

Digital Signature-Facto-DSA-ARCH-Reference Implementation/CPU_NAME

Digital Signature-Facto-DSA-ARCH-Performance Optimization Version/CPU_NAME

Digital Signature-Facto-DSA-ARCH-Resource Optimization Version/CPU_NAME

Where ARCH is x86-64, ARM or RISC-V and CPU_NAME is the name of the CPU, i.e AMD EPYC 7413. Inside each CPU_NAME folder the required details by the self-assesment guidelines is located. Here the 128,256 and 512 target bit level JSON results are found. This way we are able to maintain the naming requested by NICCS, yet including multiple CPU folders for the same architecture, measuring distincts scenarios.

For x86-64 we tested the three implementations in CPUs: Ryzen 9700x, AMD EPYC 7413, Intel Xeon E3 (Haswell). These are compatible with AVX2 extended instruction set.

For ARM we tested the three implementations in CPUs: Cortex A72, Apple M4 and Graviton3. The CPU Graviton3 is compatible with NEON and SVE, as it supports ARMV8.2-A. The CPU Cortex A72 is compatible with NEON/ASIMD as it supports ARMV8-A. Apple M4 CPU is compatible with NEON/ASIMD but not with SVE.

For RISC-V we tested the three implementations in the CPU Ky X1.

4.1.1 Implementation Differences

The three main implementations are coded in standard ISO C and don't use external libraries. But they achieve different things.

Reference Implementation Designed to be portable. It is similar to the algorithm specification pseudocode. It doesn't use instructions from the extended instruction sets AVX2/NEON/SVE. It doesn't precompute reused values, does expand monomials list naively, doesn't accumulate values for modular reductions, doesn't optimise memory allocations, between others.

The compiler flags used in this implementation is the one recommended by NICCS.

For all platforms (architectures):

`-std=c99 -O2 -Wall -Wextra -Wpedantic`

Performance Optimization Version It introduces many optimisations that aid in the public key generation, signing and verifying routines. The techniques used are: delayed accumulation for modular reductions, precomputing heavily reused values, monomial precomputed list for cubic indices, multiple architecture SIMD instructions for public matrix linear algebra operations, allocation a single big chunk to reduce memory overhead.

The implementation uses compilation flags:

`-DFACTO_SIMD_AVX2=1 -DFACTO_SIMD_NEON=1`

So different code paths exist for AVX2/NEON/SVE inside the source code, including a scalar fallback that doesn't use extended instruction sets.

The compiler flags used in this implementation are the one recommended by NICCS.

For x86-64:

`-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra.`

For ARM:

`-O3 -march=armv8.2-a+sve -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`

Resource Optimization Version The resource optimisation implementation keeps many of the performance optimization version, it uses same API and SIMD build options, but reduces peak memory by streaming the public key construction, avoids storing of large cubic intermediate matrices. It also reduces auxiliary buffers such as random pools and precomputation tables. This lowers the RAM consumption at the cost of some additional recomputation and therefore slower KeyGen compared with the performance optimisation version.

The implementation uses compilation flags:

`-DFACTO_SIMD_AVX2=1 -DFACTO_SIMD_NEON=1`

So different code paths exist for AVX2/NEON/SVE inside the source code, including a scalar fallback that doesn't use extended instruction sets.

The compiler flags used in this implementation are the one recommended by NICCS.

For x86-64:

`-Os -march=x86-64 -mavx2 -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`

For ARM:

-Os -march=armv8-a -flt0 -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra

4.1.2 Platform Differences

When dealing with multiple platform architectures, it is important to know what the cycle counter registry is. For x86-64 we use rdtsc. For ARMv8 we use pmccntr_el0. For MacOS counter is read by p_kpc.get_thread_counters. For RISC-V we use rdcycle.

For reading the cycle counter in RISC-V and MacOS, the provided test script (run-bench.sh) found in the submission must be runned as sudo or root. For accessing the cycle counter in ARMv8 it is mandatory to load a kernel module that exposes the counter to user space. Such module can be built and installed with insmod and can be found at:

https://github.com/jerinjacobk/armv8_pmu_cycle_counter_el0

For x86-64 there is no limitation. These steps must be followed, or the implementation and the test bench ngcc_bench may compile but running expects cycle counter visibility and will give an error or the cycle fields won't be populated in the JSON output report file.

4.1.3 Transmission and storage cost

For the implementation of Facto-DSA target level instances, note that reference implementation, performance and resource optimised implementation use the same byte encoding for cryptographic material. Therefore, the storage and transmission costs are the same for both implementations.

It is important to explain what these values are for the signer and verifier.

The verifier stores the public key. The signer stores the public key and the secret key.

For transmission, the public key may be sent once to a verifier. Afterwards, the signer sends the signature with each message. If the public key is sent together with a signed message, then the transmitted data is the public key, the signature and the message.

Let $|M|$ be the message length in bytes. The encoded sizes used in the submitted implementation are

$$pkBytes = 2m \binom{r+2}{3},$$

$$sigBytes = 2r,$$

and

$$skBytes = 2(r^2 + Dm + sD + N_Q(n) + N_R(n)) + 32.$$

Here, 2 is the number of bytes used to encode one element of the base field. The term 32 is the public key digest stored in the secret key.

Table 4: Storage cost in bytes for the submitted Facto-DSA instances. The verifier stores the public key. The signer stores both the public key and the secret key.

Storage scenario	Facto-DSA-128	Facto-DSA-256	Facto-DSA-512
pk stored by the verifier	40,040	456,960	5,674,240
$pk + sk$ stored by the signer	43,134	468,622	5,736,162

Table 5: Transmission cost in bytes for the submitted Facto-DSA instances. The term $|M|$ denotes the message length in bytes.

Transmission scenario	Facto-DSA-128	Facto-DSA-256	Facto-DSA-512
pk sent by the signer	40,040	456,960	5,674,240
$sig + M$ sent by the signer	$40 + M $	$68 + M $	$128 + M $
$pk + sig + M$ sent by the signer	$40,080 + M $	$457,028 + M $	$5,674,368 + M $

As a result, after public key distribution, the transmission cost involves only the signature and the message. In addition, if the verifier receives the public key together with the signed message, the public key dominates the transmission cost. A clearer view is given in the following table:

Table 6: Transmission and storage cost for the submitted Facto-DSA instances. All sizes are reported in bytes.

Item	Facto-DSA-128	Facto-DSA-256	Facto-DSA-512
q	65519	65519	65519
n	10	17	32
r	20	34	64
D	19	33	63
m	13	32	62
$s = D - m$	6	1	1
Public key storage	40,040 bytes	456,960 bytes	5,674,240 bytes
Secret key storage	3,094 bytes	11,662 bytes	61,922 bytes
Signature storage	40 bytes	68 bytes	128 bytes
Public key transmission	40,040 bytes	456,960 bytes	5,674,240 bytes
Per message signature transmission	40 bytes	68 bytes	128 bytes

Note that public key compression and private key seed expansion are not part of this submission implementation, but will be considered in future versions.

4.2 Performance test results

By following the self-assessment guidelines seen in the previous section, we execute `ngcc_bench` for each target level bit security to measure the Facto-DSA implementation versions (Reference Implementation, Performance Optimization Version and Resource Optimization Version). These are included in each test report JSON.

These indicators are collected by executing tests on the `.so` library files compiled for Facto-DSA implementations.

- Correctness. Algorithm is validated with KAT or single run of `keygen+sign+verify`.
- Clock cycles for all platforms. Previously commented (x86-64,ARM,MacOS,RISC-V)
- Throughput, operations per second, cycles per operation
- Resource consumption. Measures usage of static memory and peak memory.
- Transmission and storage cost. For Facto-DSA these are measured as bytes/op in the tables.

Only a single CPU core is utilised, with 1000 iterations per operation (keygen, sign, verify). All the runs for the three implementations passed the correctness test. A table is included per CPU tested, with all the indicators given for each target security bit level and implementation version.

The following tables report the NGCC self-assessment outputs for the submitted Facto-DSA implementation versions. Each cell gives, from top to bottom, the mean cycle count per operation, operations per second and bytes per operation. Memory columns are reported in bytes.

AMD Ryzen 7 9700X(x86-64) Clock frequency: 3600MHz. Memory: 32GB RAM. Compiler Version: clang 22.1.6. OS: Arch Linux Kernel 7.0.11

Table 7: Facto-DSA Reference implementation results on AMD Ryzen 7 9700X 8-Core Processor (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	4,141,037 916 43,134	1,490,197 2,545 64	486,432 7,793 64	77,824	77,824
256	56,344,458 68 468,622	6,821,710 556 64	5,279,432 719 64	1,105,920	69,632
512	1,231,324,705 4 5,736,162	55,055,193 69 64	65,324,300 59 64	12,185,600	69,632

Table 8: Facto-DSA Performance optimisation version results on AMD Ryzen 7 9700X 8-Core Processor (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	1,227,572	771,066	342,838	315,392	143,360
	3,089	4,918	11,055		
	43,134	64	64		
256	15,414,735	2,872,353	3,831,316	2,990,080	167,936
	247	1,321	990		
	468,622	64	64		
512	291,948,889	19,391,024	47,504,840	35,987,456	389,120
	13	196	80		
	5,736,162	64	64		

Table 9: Facto-DSA Resource optimisation version results on AMD Ryzen 7 9700X 8-Core Processor (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	6,077,342	3,753,330	400,654	40,960	40,960
	625	1,011	9,460		
	43,134	64	64		
256	87,490,060	23,154,377	4,389,826	516,096	40,960
	44	164	864		
	468,622	64	64		
512	2,282,596,240	222,074,632	54,969,829	6,004,736	40,960
	1.6616	18	69		
	5,736,162	64	64		

AMD EPYC 7413 (x86-64) Clock frequency: 2650MHz. Memory: 32GB RAM. Compiler Version: clang 22.1.6. OS: Arch Linux Kernel 7.0.11

Table 10: Facto-DSA Reference implementation results on AMD EPYC 7413 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	5,246,837 506 43,134	1,489,816 1,779 64	544,615 4,863 64	77,824	77,824
256	69,850,385 38 468,622	7,244,582 366 64	6,038,055 439 64	1,105,920	69,632
512	1,538,985,104 1.7219 5,736,162	57,673,065 46 64	73,747,235 36 64	12,185,600	69,632

Table 11: Facto-DSA Performance optimisation version results on AMD EPYC 7413 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	1,647,062 1,609 43,134	931,452 2,844 64	420,228 6,302 64	307,200	135,168
256	21,073,893 126 468,622	3,368,975 787 64	4,711,235 563 64	2,990,080	167,936
512	398,461,718 7 5,736,162	21,363,696 125 64	58,178,480 46 64	35,983,360	385,024

Table 12: Facto-DSA Resource optimisation version results on AMD EPYC 7413 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	7,451,711	3,640,131	458,633	40,960	40,960
	356	728	5,774		
	43,134	64	64		
256	107,232,369	22,482,540	5,123,797	516,096	40,960
	25	118	518		
	468,622	64	64		
512	2,592,306,764	233,816,749	64,923,022	6,004,736	40,960
	1.0223	12	41		
	5,736,162	64	64		

XEON E3-1220 (x86-64) Clock frequency: 3100MHz. Memory: 4GB RAM. Compiler Version: clang 21.1.8. OS: Debian Kernel 7.0.12

Table 13: Facto-DSA Reference implementation results on XEON E3-1220 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	7,664,892	2,440,880	857,300	73,728	73,728
	314	983	2,798		
	43,134	64	64		
256	104,970,821	13,171,202	9,620,561	1,105,920	69,632
	23	183	250		
	468,622	64	64		
512	2,313,589,940	105,287,185	127,873,255	12,185,600	69,632
	1.0373	23	19		
	5,736,162	64	64		

Table 14: Facto-DSA Performance optimisation version results on XEON E3-1220 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	3,024,043	1,308,737	742,217	327,680	155,648
	794	1,833	3,231		
	43,134	64	64		
256	38,474,165	6,782,550	8,249,714	2,981,888	159,744
	63	354	291		
	468,622	64	64		
512	831,912,710	48,059,344	104,008,777	35,966,976	368,640
	3	50	24		
	5,736,162	64	64		

Table 15: Facto-DSA Resource optimisation version results on XEON E3-1220 (x86-64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	15,398,091 156 43,134	4,788,098 502 64	741,083 3,236 64	40,960	40,960
256	229,092,486 11 468,622	30,429,514 79 64	8,757,023 275 64	516,096	40,960
512	5,891,107,244 0.4074 5,736,162	290,332,127 9 64	106,892,710 23 64	6,004,736	40,960

Graviton3 (aarch64) Clock frequency: 2600MHz. Memory: 4GB RAM. Compiler Version: clang 15.0.7. OS: Amazon Linux Kernel 6.1.174

Table 16: Facto-DSA Reference implementation results on Graviton3 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	4,406,447	1,047,361	592,769	270,336	270,336
	589	2,469	4,349		
	43,134	64	64		
256	59,513,064	5,007,281	6,667,373	1,302,528	270,336
	44	519	390		
	468,622	64	64		
512	1,285,741,640	41,499,179	83,475,324	12,386,304	270,336
	3	63	32		
	5,736,162	64	64		

Table 17: Facto-DSA Performance optimisation version results on Graviton3 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	2,109,364	1,030,936	554,070	450,560	278,528
	1,229	2,509	4,650		
	43,134	64	64		
256	28,446,411	4,836,410	6,250,486	3,076,096	307,200
	92	537	416		
	468,622	64	64		
512	587,453,732	27,610,468	77,641,134	36,134,912	536,576
	5	95	34		
	5,736,162	64	64		

Table 18: Facto-DSA Resource optimisation version results on Graviton3 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	8,263,380	1,637,837	547,490	266,240	266,240
	315	1,582	4,706		
	43,134	64	64		
256	125,759,238	8,897,697	6,117,433	737,280	266,240
	21	292	425		
	468,622	64	64		
512	3,381,150,543	83,137,107	75,815,244	6,225,920	266,240
	0.7682	32	35		
	5,736,162	64	64		

Cortex-A72 (aarch64) Clock frequency: 1500MHz. Memory: 4GB RAM. Compiler Version: clang 19.1.7. OS: Linux Raspberrypi Kernel 6.18.34

Table 19: Facto-DSA Reference implementation results on Cortex-A72 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	10,131,062 178 43,134	2,346,053 767 64	1,211,694 1,485 64	204,800	139,264
256	142,865,721 13 468,622	12,512,616 144 64	14,054,664 129 64	1,171,456	139,264
512	3,057,669,651 0.5887 5,736,162	105,005,872 18 64	176,929,052 11 64	12,255,232	139,264

Table 20: Facto-DSA Performance optimisation version results on Cortex-A72 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	4,820,965 374 43,134	2,301,417 782 64	1,042,601 1,726 64	319,488	147,456
256	69,343,785 26 468,622	11,735,263 154 64	11,505,712 157 64	2,945,024	176,128
512	1,462,090,761 1.2311 5,736,162	93,618,336 20 64	142,839,303 13 64	36,003,840	405,504

Table 21: Facto-DSA Resource optimisation version results on Cortex-A72 (aarch64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	21,057,435 86 43,134	2,868,402 628 64	1,033,262 1,741 64	200,704	135,168
256	343,448,671 6 468,622	17,349,381 104 64	11,629,630 155 64	606,208	135,168
512	10,130,069,391 0.1777 5,736,162	170,870,870 11 64	144,095,237 13 64	6,094,848	135,168

Apple MacBook Pro (arm64) Clock frequency: 4500Mhz. Memory: 24GB. Compiler Version: clang 17.0.0. OS: Darwin Kernel 25.5.0

Remark For Apple OS tests, Static and Peak memory values were not collected as nggc.bench program has non POSIX code to measure these. It is not adapted for other non Linux OS.

Table 22: Facto-DSA Reference implementation results on Apple MacBook Pro (arm64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op
128	3,195,864	1,059,581	499,817
	1,258	4,067	8,184
	43,134	64	64
256	42,857,758	4,424,497	5,681,989
	95	961	723
	468,622	64	64
512	940,562,848	26,883,320	71,383,028
	4	149	56
	5,736,162	64	64

Table 23: Facto-DSA Performance optimisation version results on Apple MacBook Pro (arm64).
Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op
128	1,515,910	1,094,377	450,346
	2,524	3,800	8,951
	43,134	64	64
256	18,916,741	4,448,331	5,173,002
	213	925	781
	468,622	64	64
512	413,893,745	23,993,452	63,070,404
	10	169	64
	5,736,162	64	64

Table 24: Facto-DSA Resource optimisation version results on Apple MacBook Pro (arm64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op
128	4,883,469	1,182,778	461,235
	813	3,345	8,692
	43,134	64	64
256	72,210,203	6,737,384	5,285,602
	56	597	768
	468,622	64	64
512	1,939,567,041	60,054,927	64,517,180
	2	67	63
	5,736,162	64	64

Ky X1 RISCv (riscv64) Clock frequency: 1600MHz. Memory: 8GB RAM. Compiler Version: clang 18.1.3. OS: Orange Pi RV2 Linux Kernel 6.6.63, Ubuntu 24.04 Based

Table 25: Facto-DSA Reference implementation results on Ky(R) X1 processor (riscv64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	45,788,599 35 43,134	9,884,838 162 64	3,113,792 511 64	40,960	40,960
256	636,986,678 3 468,622	80,930,042.2 19.762 64	36,721,048 44 64	1,077,248	45,056
512	14,737,213,194 0.109 5,736,162	702,352,670 3 64	469,500,662 4 64	12,156,928	40,960

Table 26: Facto-DSA Performance optimisation version results on Ky(R) X1 processor (riscv64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	16,336,692 98 43,134	10,560,394 151 64	1,995,124 795 64	249,856	77,824
256	243,501,850 6 468,622	57,850,264 28 64	22,210,538 72 64	2,875,392	106,496
512	5,672,315,024 0.282 5,736,162	1,091,160,221 1.466 64	273,707,443 6 64	35,934,208	335,872

Table 27: Facto-DSA Resource optimisation version results on Ky(R) X1 processor (riscv64). Operation cells list cycles/op, ops/s and bytes/op.

Target bits	KeyGen cycles/op ops/s bytes/op	Sign cycles/op ops/s bytes/op	Verify cycles/op ops/s bytes/op	Peak memory bytes	Static memory bytes
128	75,921,225 21 43,134	7,981,283 200 64	2,638,885 602 64	32,768	32,768
256	1,449,446,070 1.104 468,622	118,116,932 14 64	29,519,500 54 64	503,808	32,768
512	49,032,343,824 0.033 5,736,162	725,065,648 2 64	362,313,097 5 64	5,992,448	32,768

Testing acknowledgments Thanks to my colleague Jose Angel Gumiel for lending his RISC-V based Orange Pi RV2 board for testing all the implementations and by providing C code to read the RISC-V cycle counters, which was adapted for the `ngcc_bench` program. Thanks to Alejandro Quiroga for lending his ARM based Apple Macbook Pro for testing the implementations.

5 Feature Statements

The claims are only limited to the submitted construction and to the implementations found in the package.

Feature	Statement
Innovativeness	Cubic schemes on Multivariate Public Key Cryptography are not that common as quadratic. Factorisation based schemes and primitives are a potential area of study, specially with the submitted cubic construction.
Simplicity	MPKC based schemes are very simple in terms of their construction. Only linear, polynomial and modular algebra is needed and its implementation can be made portable not dependant of external libraries.
Flexibility	The parameters n , m , s and $R_{\max}$ can be changed by extending the design rule. Larger values raise the cost of direct solving and structural attacks. The proposal gives short signatures and large public keys, which is a common trade off in multivariate signatures.
Compatibility	The reference implementation is written in ISO C and does not require an external library. It can be compiled as source code with a standard C toolchain. The public interface (API) KeyGen, Sign and Verify works on multiple platforms.
Extensibility	The prime field, the output minus dimension, the hidden triangular family and the retry bound can be changed in later parameter sets if public review gives new cost data.
Advantages on variant platforms	The Performance and Resource optimization implementation version do use AVX2 for x86-64 and NEON/SVE for ARMv8-A and ARMV8.2-A platforms. The use of extended instruction sets achieves a notable speedup in keygen and signature verification, as seen in test results

References

- [1] Institute of Commercial Cryptography Standards. *Submission Requirements for Public-Key Cryptographic Algorithms*. NGCC public key algorithm call, October 2025. <https://www.niccs.org.cn/niccs/Notice/1975896137741635584/1Dop1mav.pdf>
- [2] The MAYO Team. *The MAYO Signature Scheme Specification*. Public specification for the NIST additional post quantum signatures process.
- [3] The QR-UOV Team. *QR-UOV Specification, Round 2*. Public specification for the NIST additional post quantum signatures process.
- [4] Haruhisa Kosuge and Keita Xagawa. Probabilistic Hash-and-Sign with Retry in the Quantum Random Oracle Model. IACR Cryptology ePrint Archive, Paper 2022/1359, 2022.
- [5] Christopher Wolf and Bart Preneel. Taxonomy of Public Key Schemes Based on the Problem of Multivariate Quadratic Equations. IACR Cryptology ePrint Archive, Paper 2005/077, 2005.
- [6] Jintai Ding and Bo-Yin Yang. Multivariate Public Key Cryptography. In *Post-Quantum Cryptography*, Springer, 2009.
- [7] Jacques Patarin. Hidden Fields Equations and Isomorphisms of Polynomials. EUROCRYPT, 1996.
- [8] Jean-Charles Faugère. A New Efficient Algorithm for Computing Groebner Bases. *Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.
- [9] Jean-Charles Faugère. A New Efficient Algorithm for Computing Groebner Bases Without Reduction to Zero. ISSAC, 2002.
- [10] Aviad Kipnis, Jacques Patarin and Louis Goubin. Unbalanced Oil and Vinegar Signature Schemes. EUROCRYPT, 1999.
- [11] John Baena, Pierre Briaud, Daniel Cabarcas, Ray Perlner, Daniel Smith-Tone and Javier Verbel. Improving Support-Minors Rank Attacks: Applications to GeMSS and Rainbow. CRYPTO, 2022.
- [12] Yasufumi Hashimoto. On the Security of Cubic UOV and Its Variants. IACR Cryptology ePrint Archive, Paper 2016/788, 2016.
- [13] Dustin Moody, Ray Perlner and Daniel Smith-Tone. Key Recovery Attack on the Cubic ABC Simple Matrix Multivariate Encryption Scheme. 2016.
- [14] Pierre Pébereau. One vector to rule them all: Key recovery from one vector in UOV schemes. PQCrypto 2024, LNCS 14772, pp. 92–108, 2024.
- [15] Irene Di Muzio, Martin Feussner and Igor Semaev. Multivariate Signatures with Polynomial Factorization. IACR Cryptology ePrint Archive, Paper 2025/2061, 2025.
- [16] Alexandre Camelin, Thai Hung Le, Brice Minaud, Phong Q. Nguyen and Florian Tounshakhoff. X24 Down: Cryptanalysis of Hankel Based Multivariate Signatures. IACR Cryptology ePrint Archive, Paper 2026/803, 2026.
- [17] Borja Gómez. Factorisation Based Multivariate Schemes: Structural Properties and New Constructions. IACR Cryptology ePrint Archive, Paper 2026/797, 2026.